

Latent Collaboration in Multi-Agent Systems

Jiaru Zou^{1,2,*}, Xiyuan Yang^{2,*}, Ruizhong Qiu^{2,†}, Gaotang Li^{2,†}, Katherine Tieu^{2,†}, Pan Lu^{3,†}, Ke Shen, Hanghang Tong², Yejin Choi³, Jingrui He^{2,✉}, James Zou^{3,✉}, Mengdi Wang^{1,✉}, Ling Yang^{1,✉}

¹Princeton University ²University of Illinois Urbana-Champaign ³Stanford University

*Co-Leadership †Core Contributors ✉Corresponding Authors

Code: <https://github.com/Gen-Verse/LatentMAS>

Multi-agent systems (MAS) extend large language models (LLMs) from independent single-model reasoning to coordinative system-level intelligence. While existing LLM agents depend on text-based mediation for reasoning and communication, we take a step forward by enabling models to collaborate directly within the continuous latent space. We introduce LatentMAS, an end-to-end training-free framework that enables pure latent collaboration among LLM agents. In LatentMAS, each agent first performs auto-regressive latent thoughts generation through last-layer hidden embeddings. A shared latent working memory then preserves and transfers each agent’s internal representations, ensuring lossless information exchange. We provide theoretical analyses establishing that LatentMAS attains higher expressiveness and lossless information preservation with substantially lower complexity than vanilla text-based MAS. In addition, empirical evaluations across 9 comprehensive benchmarks spanning math and science reasoning, commonsense understanding, and code generation show that LatentMAS consistently outperforms strong single-model and text-based MAS baselines, achieving up to 14.6% higher accuracy, reducing output token usage by 70.8%-83.7%, and providing 4×-4.3× faster end-to-end inference. These results demonstrate that our new latent collaboration framework enhances system-level reasoning quality while offering substantial efficiency gains without any additional training.

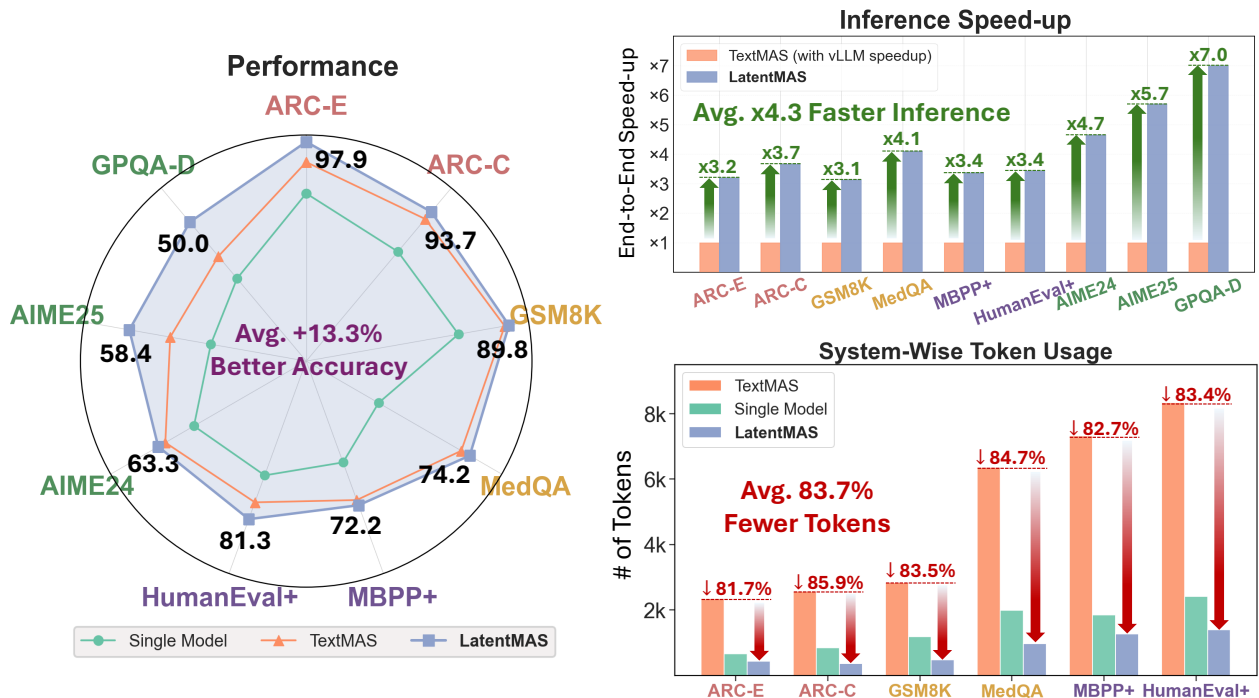


Figure 1 | Evaluation of LatentMAS across (i) accuracy performance (%), (ii) inference speed (times(s)/run), and (iii) token usage (per token) over 9 benchmarks and 3 LLM model scales under the Hierarchical MAS setting. LatentMAS consistently improves system-level reasoning accuracy while substantially reducing computational overhead compared with single model and text-based MAS.

1. Introduction

Model collaboration emerges as the foundation of system-level intelligence in the era of Agentic AI (Acharya et al., 2025). Recent advances in multi-agent systems (MAS) (Hong et al., 2023; Hu et al., 2025; Wu et al., 2024) have catalyzed a paradigm shift from solitary, model-centric reasoning into a collaborative endeavor among multiple interacting models. Among these, large language model (LLM)-based MAS has been adopted across various downstream applications, including cooperative math and science reasoning (Pezeshkpour et al., 2024; Zhou et al., 2025), distributed tool-use in open-domain QA (Jin et al., 2025; Li et al., 2025d), and embodied decision-making in robotics (Feng et al., 2025; Li et al., 2025c). Within LLM-based MAS, natural language or text generally serves as the *lingua franca*—the common medium that carries each agent’s internal thoughts and enables communication across different agents (Guo et al., 2024).

Beyond explicit text, several studies have explored the use of LLMs’ continuous latent space as a new form of “model language,” (Chen et al., 2025b) by either (i) leveraging hidden representations within transformers to enable single model’s internal latent chain-of-thought (CoT) reasoning (Hao et al., 2024; Zhang et al., 2025; Zheng et al., 2025), or (ii) employing KV caches or layer embeddings for information exchange across two models (Fu et al., 2025; Liu et al., 2024). However, a comprehensive model collaboration framework unifying both latent reasoning and latent communication remains unexplored. Moving one step forward, we investigate:



Can multi-agent systems achieve pure latent collaboration?

To address this question, we introduce **LatentMAS**, an end-to-end collaborative framework that operates entirely within the continuous latent space. Our core design integrates both internal *latent thoughts generation* and cross-agent *latent working memory transfer*. Inside each agent, reasoning unfolds through auto-regressive generation of last-layer hidden representations, capturing the model’s ongoing internal thoughts without explicit decoding. Across agents, information is exchanged via shared latent working memory stored in layer-wise KV caches, capturing both the input context and newly generated latent thoughts. Overall, LatentMAS is completely *training-free*, enabling all agents to think and interact purely through their internal latent representations.

Building on our framework design, LatentMAS is grounded on three foundational principles, verified by comprehensive theoretical and empirical analyses:

- **Reasoning Expressiveness:** Hidden representations naturally encode models’ continuous thoughts, allowing each latent step to convey far richer information than discrete tokens.
- **Communication Fidelity:** Latent working memory preserves input representations and latent thoughts of each model, enabling lossless cross-agent information transfer.
- **Collaboration Complexity:** LatentMAS achieves higher collaborative expressiveness of TextMAS while achieving significantly lower inference complexity.

The first two principles jointly underscore the advantage of LatentMAS by enabling richer latent reasoning and lossless latent communication. The third principle further provides an overall complexity analysis, showing that LatentMAS achieves substantially lower computational complexity than text-based MAS while maintaining a higher level of model expressiveness.

To empirically assess the efficacy of LatentMAS, we conduct comprehensive evaluations on nine benchmarks spanning math and science reasoning, commonsense understanding, and code generation, as illustrated in Figure 1. Across both sequential and hierarchical MAS settings and three backbone scales (4B, 8B, and 14B (Yang et al., 2025)), LatentMAS consistently outperforms strong single-model and text-based MAS baselines by (i) improving accuracy by up to 14.6%, (ii) reducing output

token usage by 70.8%-83.7%, and (iii) delivering $4\times$ - $4.3\times$ faster end-to-end inference. These results demonstrate that latent collaboration not only enhances system-level reasoning quality but also provides substantial efficiency gains without any additional training. Further detailed analyses of latent thought expressiveness, working-memory transfer, and input-output alignment confirm that LatentMAS enables semantically meaningful, lossless, and stable collaboration entirely in latent space.

2. Preliminary and Notations

Auto-regressive Generation in Transformer. Let $f_\theta(\cdot)$ denotes the function computed by a standard Transformer model (Vaswani et al., 2017), parameterized by θ . Given an input sequence $x = (x_1, x_2, \dots, x_T)$, the transformer $f_\theta(\cdot)$ first encodes each token via its input embedding layer W_{in} to obtain token embeddings up to step t , i.e., $E = [e_1, e_2, \dots, e_t] \in \mathbb{R}^{t \times d_h}$, where d_h is the model’s hidden dimension. The input token embeddings E then successively process through L transformer layers in the forward pass through the model’s residual stream, yielding the final-layer hidden representations $H = [h_1, h_2, \dots, h_t] \in \mathbb{R}^{t \times d_h}$. For next token generation, the model computes:

$$f_\theta(x_{t+1} | x_{\leq t}) = \text{softmax}(h_t W_{\text{out}}), \quad (1)$$

where W_{out} denotes the language model head that maps the hidden representation to the vocabulary space. Each token is generated in an auto-regressive manner and appended to the input sequence. For *latent space generation*, the model feeds the last hidden state from the previous token directly as the next input embedding without explicit decoding (Hao et al., 2024; Zhu et al., 2025).

KV Cache as Working Memory. In decoder-only Transformers, the Key-Value (KV) cache functions as a dynamic working memory during auto-regressive generation, storing intermediate representations from previous decoding steps to avoid redundant computation. Specifically, given the input embeddings E , each transformer layer projects them through projection matrices W_Q, W_K, W_V to obtain Q, K, V . When the next token at step $t + 1$ is generated, the model appends its embedding to the input sequence and updates the cache ($K_{\text{cache}}, V_{\text{cache}}$) as:

$$K_{\text{cache}} \leftarrow [K_{\leq t}; K_{t+1}], \quad V_{\text{cache}} \leftarrow [V_{\leq t}; V_{t+1}], \quad (2)$$

where $K_{\leq t}, V_{\leq t}$ are accumulated key/value matrices from all previous steps and K_{t+1}, V_{t+1} are new key/value vectors computed from the current token’s hidden state. This accumulative property enables the KV cache to maintain a growing working memory of model internal representations.

LLM-based MAS Setting. We consider a multi-agent system S composed of N agents, denoted as $\mathcal{A} = \{A_1, A_2, \dots, A_N\}$, where each agent A_i is an LLM corresponding to f_{θ_i} above. At inference time, an input question q is provided to the system S , which orchestrates interactions among agents to collaboratively produce a final answer a corresponding to q . As MAS design paradigms are not definitive in general and often vary across downstream tasks (Cemri et al., 2025; Tran et al., 2025), we do not restrict our latent collaboration design to any particular architecture. Instead, we adopt two most commonly used MAS settings (*sequential* and *hierarchical*) as the bases to experimentally evaluate our method.

Figure 2 illustrates the two MAS architecture settings. In the **sequential MAS**, we adopt a chain-of-agents design (Zhang et al., 2024b; Zhao et al., 2025a) comprising four LLM agents: planner, critic, refiner, and solver. These agents assume complementary reasoning roles and are organized in a sequential pipeline,

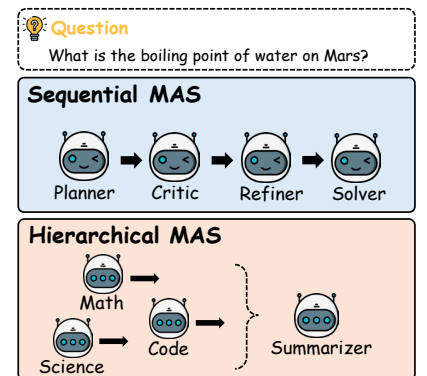


Figure 2 | Illustration of sequential and hierarchical MAS.

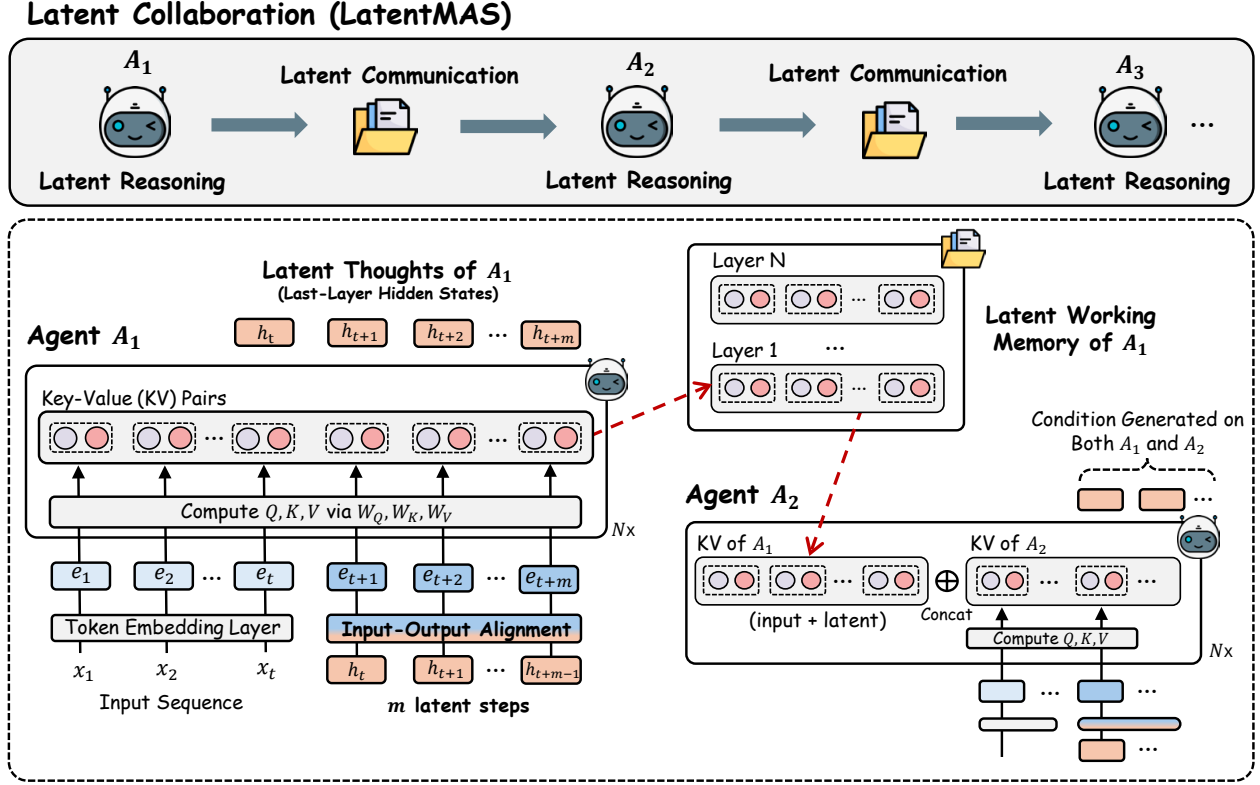


Figure 3 | **Overview of LatentMAS.** Each LLM agent in the system first generates latent thoughts through last-layer hidden states, then transfers information layer-wise via shared latent working memory stored in KV-caches, enabling completely system-wide latent collaboration.

where the CoT output of each agent with the question q serves as the input to the next agent. In the **hierarchical MAS**, we adopt a domain-specialized design (Zhao et al., 2025b; Zhuge et al., 2024). Multiple LLM agents, including code, math, and science agents, operate as different domain experts. Each agent independently reasons over the question q from its disciplinary perspective. A summarizer agent then receives all intermediate responses along with the question q and performs hierarchical aggregation to synthesize and refine the final answer.

3. Building a Latent Collaborative Multi-Agent System

We introduce LatentMAS, an end-to-end latent collaboration framework that, given an input question, all agents reason and communicate entirely within the latent space and only decode the final answer in text. Our method enables LLM agents within the system to (i) perform super-expressive thoughts generation in the latent space (Section 3.1), (ii) preserve and transfer each agent’s latent working memory with lossless fidelity across interactions (Section 3.2), and (iii) achieve substantially lower complexity than vanilla text MAS while maintaining the same level of expressiveness (Section 3.3).

Method Roadmap. In following sections, we present the complete pipeline of LatentMAS, detailing each component and interleaving theoretical analyses to justify the corresponding design principles.

3.1. Auto-regressive Latent Thoughts Generation in Agents.

We start by describing, inside each LLM agent, how the model performs latent reasoning through its layer-wise hidden states. Instead of generating explicit tokens, reasoning unfolds directly within the model by auto-regressively appending hidden representations produced by the final transformer layer.

Specifically, given the input embeddings $E = [e_1, e_2, \dots, e_t]$ containing the information from the question q and each agent’s instruction prompt, each LLM agent $A_i \in \mathcal{A}$ passes E through L transformer layers to compute the last-layer hidden representation h_t at current step t . Then, we insert h_t as the input embedding for the next step $t + 1$, replacing the original decoding and next-token embedding processes used in standard token generation. We auto-regressively repeat the process for m latent steps, yielding a sequence of newly generated last-layer hidden states $H = [h_{t+1}, h_{t+2}, \dots, h_{t+m}]$. We define the continuous output representations H as the *latent thoughts* generated by A_i .

Input-Output Distribution Alignment. Since the newly generated H form a sequence of dense, high-level representations, directly inserting them into shallow layers as input embeddings may lead to out-of-distribution activations (Meegahapola et al., 2019; Zhou et al., 2019), as these hidden states differ from the statistical patterns of learned token embeddings. To mitigate this in a training-free manner, we propose a *linear alignment operator* that maps last-layer hidden states back to the valid input embeddings. Specifically, given $W_{\text{in}}, W_{\text{out}}$ as the input and output embedding layers of A_i , we seek a projection matrix $W_a \in \mathbb{R}^{d_h \times d_h}$ that maps each output vector $h \in H$ to a new input vector e to align with valid input space defined by W_{in} :

$$e = hW_a, \quad \text{where } W_a \approx W_{\text{out}}^{-1}W_{\text{in}}, \quad (3)$$

where the W_{out}^{-1} is the pseudo-inverse of W_{out} ¹. We then append the aligned vector e into the input sequence for auto-regressive latent generation. Note that W_a is a small projection matrix of size $d_h \times d_h$ (e.g., $d_h = 1024$ for Qwen3-0.6B) and is computed once and reused in all subsequent latent steps. This design makes the alignment computationally negligible while maintaining distributional consistency between latent and discrete representations. In Appendix A.2, we further provide a detailed theoretical justification for the effectiveness of W_a during the input-output alignment process.

Expressiveness on Continuous Latent Thoughts. With the mechanism of latent thought generation established within each agent, we next provide a theoretical analysis to quantify its representational advantage over conventional discrete token generation. The following theorem formalizes that latent thoughts, which inherently preserve richer semantic structures, achieve substantially higher expressive capacity than discrete text-based reasoning.

Theorem 3.1 (Expressiveness of Latent Thoughts). *Under the Linear Representation Hypothesis on h (detailed in Assumption B.1), if the sequence of all latent thoughts with length m can be expressed losslessly through corresponding text-based reasoning, then the length of text (in tokens) needs to be at least $\Omega(d_h m / \log |\mathcal{V}|)$, where $|\mathcal{V}| > 1$ denotes the vocabulary size.*

Remark 3.2. Theorem 3.1 suggests that latent thoughts generation can be $O(d_h / \log |\mathcal{V}|)$ times more efficient than text-based reasoning. In addition, the expressiveness scales linearly with d_h , implying that larger models inherently exhibit greater latent reasoning capacity.

As an illustration to Remark 3.2, for Qwen3-4B / 8B / 14B models (Yang et al., 2025), latent thoughts generation can be 235.7 / 377.1 / 471.4 times more efficient than text-based reasoning. The full proof of Theorem 3.1 is provided in Appendix B.1. Beyond reasoning within individual agents, collaboration in LatentMAS further relies on how these agents exchange latent information, which we detail next.

3.2. Working Memory Preservation and Thoughts Transfer across Agents.

In text-based MAS, after one LLM agent completes its generation, the natural language output is directly appended to the input sequence of the next agent. However, since each agent in LatentMAS

¹As W_{out} is typically non-square, its true inverse cannot be directly calculated as is. In practice, we compute W_a in Equation 3 by solving a ridge regression (Hoerl and Kennard, 1970): $\min_{W_a} \{\|W_{\text{out}}W_a - W_{\text{in}}\|_F^2 + \lambda \|W_a\|_F^2\}$, which can be computed efficiently in polynomial complexity by $W_a = (W_{\text{out}}^\top W_{\text{out}} + \lambda I)^{-1} W_{\text{out}}^\top W_{\text{in}}$ (Detailed in A.1).

performs hidden-state generation without explicit text outputs, we design a new **latent working memory transfer mechanism** to ensure lossless information preservation and exchange.

For clarity, we describe the transfer mechanism using the first two consecutive LLM agents $A_1, A_2 \in \mathcal{A}$ in LatentMAS. As shown in Figure 3, agent A_1 first performs m latent steps of generation (Section 3.1). After completing these steps, we extract the KV-caches from all L transformer layers of A_1 once, and define its *latent working memory* as:

$$\mathcal{M}_{A_1} = \left\{ \left(K_{A_1, \text{cache}}^{(l)}, V_{A_1, \text{cache}}^{(l)} \right) \mid l = 1, 2, \dots, L \right\}, \quad (4)$$

$$\text{where } K_{A_1, \text{cache}}^{(l)} = [K_{A_1, 1}^{(l)}, \dots, K_{A_1, t+m}^{(l)}], \quad V_{A_1, \text{cache}}^{(l)} = [V_{A_1, 1}^{(l)}, \dots, V_{A_1, t+m}^{(l)}].$$

Here $K_{A_1, \text{cache}}^{(l)}$ and $V_{A_1, \text{cache}}^{(l)}$ are accumulated key and value matrices at the l -th layer. Unlike existing cache-sharing methods (Fu et al., 2025; Ye et al., 2025a) that exchange information only on prefilled input context across models, the collection of layer-wise caches in $\mathcal{M}_{A_1}$ encapsulates both the initial input context and the newly generated latent thoughts of agent A_1 .

Next, the successive agent A_2 integrates the working memory $\mathcal{M}_{A_1}$ from agent A_1 . Before A_2 generates latent thoughts (i.e., last-layer hidden states), we perform layer-wise concatenation to update its KV cache by prepending each $K_{A_1, \text{cache}}^{(l)}$ and $V_{A_1, \text{cache}}^{(l)}$ to existing $K_{A_2, \text{cache}}^{(l)}$ and $V_{A_2, \text{cache}}^{(l)}$. By doing so, the new latent thoughts generation in A_2 is conditioned on both the working memory of A_1 and its own internal representations.

Lossless Information Transfer. The latent working memory transfer mechanism ensures that each succeeding agent in LatentMAS seamlessly receives its predecessor’s complete output without re-encoding. The following theorem formalizes this property, showing that latent working memory transfer guarantees information fidelity equivalent to explicit input exchange.

Theorem 3.3 (Information Preservation via Latent Working Memory). *In both latent and text-based reasoning, the outputs of an agent when receiving latent working memory from preceding agents are equivalent to those obtained when directly inputting the preceding agents’ outputs.*

The full proof of Theorem 3.3 is provided in B.2. In addition, with lossless information preservation, we transfer latent working memory in KV rather than directly transmitting hidden states to avoid redundant recomputation for the successive agent.

3.3. End-to-End Pipeline with Complexity Analyses

For the remaining agents in LatentMAS, we follow the same latent thoughts generation and working memory transfer mechanism described above. Specifically, agent A_3 inherits the working memory $\mathcal{M}_{A_2}$ from the preceding agent A_2 , performs auto-regressive last-layer hidden state generation, and subsequently transmits its updated latent working memory $\mathcal{M}_{A_3}$ to the next agent. This process continues across all agents in LatentMAS, with only the last agent decoding the final answer. Below, we theoretically analyze the overall complexity of our framework.

Theorem 3.4 (LatentMAS Complexity). *The time complexity for each agent of LatentMAS is $O((d_h^2 m + d_h m^2 + d_h t m)L)$, where t is the input length of this agent, and m is the length of latent thoughts. In contrast, assuming Theorem 3.1, the time complexity for each agent of the vanilla text-based MAS needs to be $O((d_h^3 m \frac{1}{\log |\mathcal{V}|} + d_h^3 m^2 \frac{1}{\log^2 |\mathcal{V}|} + d_h^2 t m \frac{1}{\log |\mathcal{V}|})L + d_h^2 |\mathcal{V}| m \frac{1}{\log |\mathcal{V}|})$ to achieve the same expressiveness.*

Proof of Theorem 3.4 is provided in B.3. Note that LatentMAS is agnostic to specific model collaboration strategies and can be seamlessly applied to sequential, hierarchical, or other advanced MAS designs.

Table 1 | **Main results of LatentMAS on 6 general tasks under the Sequential MAS setting.** We report 3 metrics in total, including task accuracy (% , “**Acc.**”), total output token usage (“**Token**”), and end-to-end inference speed (time(s) / run, “**Speed**”). We compare LatentMAS with both TextMAS and single-model (“Single”) baselines. For each metric, we **bold** the better performance and visualize LatentMAS gains over TextMAS in the **Improve** columns.

Tasks	Metrics	Qwen3-4B			Improve	Qwen3-8B			Improve	Qwen3-14B			Improve
		Single	TextMAS	LatentMAS		Single	TextMAS	LatentMAS		Single	TextMAS	LatentMAS	
Sequential MAS Setting													
ARC-E	Acc.	95.4	96.4	98.6	↑ 2.2	95.6	99.1	98.8	↓ 0.3	97.2	99.0	99.4	↑ 0.4
	Token	724	2420	581	↓ 76.0%	656	2085	490	↓ 76.5%	608	1670	224	↓ 86.6%
	Speed	369	2874	512	×5.6	404	3702	1759	×2.1	551	9171	2124	×4.3
ARC-C	Acc.	89.2	90.0	92.3	↑ 2.3	91.0	94.6	94.4	↓ 0.2	92.6	95.9	95.6	↓ 0.3
	Token	913	2678	718	↓ 73.2%	846	2252	529	↓ 76.5%	773	2985	426	↓ 85.7%
	Speed	97	1579	260	×6.1	266	2059	703	×2.9	338	5125	1136	×4.5
GSM8K	Acc.	82.4	89.8	88.2	↓ 1.6	81.1	92.3	93.8	↑ 1.5	83.7	93.8	95.2	↑ 1.4
	Token	1136	3172	607	↓ 80.9%	1280	2324	860	↓ 63.0%	1118	3324	644	↓ 80.6%
	Speed	469	1970	375	×5.3	449	1739	543	×3.2	536	3729	1952	×1.9
MedQA	Acc.	47.7	65.3	66.3	↑ 1.0	53.0	75.0	75.3	↑ 0.3	64.7	80.3	80.7	↑ 0.4
	Token	2134	3962	1685	↓ 57.5%	2098	4260	1555	↓ 63.5%	1746	3444	1841	↓ 46.5%
	Speed	236	1267	438	×2.9	476	1923	928	×2.1	1360	4142	1420	×2.9
MBPP+	Acc.	63.5	69.8	73.5	↑ 3.7	64.8	69.5	74.6	↑ 5.1	68.5	72.8	75.7	↑ 2.9
	Token	1634	4420	1339	↓ 69.7%	2053	3695	1164	↓ 68.5%	1858	4971	1621	↓ 67.4%
	Speed	523	2148	577	×3.7	1064	3628	1275	×2.8	2410	8728	2400	×3.6
HumanEval+	Acc.	75.0	79.7	79.9	↑ 0.2	74.4	80.5	80.5	↑ 0.0	76.8	81.1	86.5	↑ 5.4
	Token	2380	5987	1775	↓ 70.4%	2507	4593	1866	↓ 59.4%	2366	5934	2042	↓ 65.6%
	Speed	274	1044	350	×3.0	502	1619	497	×3.3	1084	4062	1285	×3.2

Table 2 | **Main results of LatentMAS on 6 general tasks under the Hierarchical MAS setting.** We report accuracy, token usage, and end-to-end speed, and highlight the performance gains following the same evaluation protocol as in Table 1.

Tasks	Metrics	Qwen3-4B				Improve	Qwen3-8B				Improve	Qwen3-14B				Improve
		Single	TextMAS	LatentMAS			Single	TextMAS	LatentMAS			Single	TextMAS	LatentMAS		
Hierarchical MAS Setting																
ARC-E	Acc.	95.4	97.1	96.8	↓ 0.3	95.6	98.2	98.3	↑ 0.1	97.2	98.3	98.7	↑ 0.4			
	Token	724	2054	363	↓ 82.3%	656	2237	308	↓ 86.2%	608	2752	619	↓ 77.5%			
	Speed	369	2239	591	×3.8	404	3619	1779	×2.0	551	7102	1884	×3.8			
ARC-C	Acc.	89.2	92.5	91.7	↓ 0.8	91.0	93.3	93.9	↑ 0.6	92.6	95.3	95.5	↑ 0.2			
	Token	913	2674	447	↓ 83.3%	846	2854	344	↓ 87.9%	773	2167	295	↓ 86.4%			
	Speed	97	1275	299	×4.3	266	2034	714	×2.8	338	4283	1090	×3.9			
GSM8K	Acc.	82.4	89.4	88.4	↓ 1.0	81.1	90.4	89.5	↓ 0.9	83.7	90.8	91.6	↑ 0.8			
	Token	1136	3098	555	↓ 82.1%	1280	2370	353	↓ 85.1%	1118	3021	495	↓ 83.6%			
	Speed	469	1878	360	×5.2	449	1365	702	×1.9	536	3675	1631	×2.3			
MedQA	Acc.	47.7	65.0	67.3	↑ 2.3	53.0	76.3	77.0	↑ 0.7	64.7	78.0	78.3	↑ 0.3			
	Token	2134	6702	1015	↓ 84.9%	2098	6893	1007	↓ 85.4%	1746	5473	899	↓ 83.6%			
	Speed	236	1495	557	×2.7	476	3387	964	×3.5	1360	7591	1250	×6.1			
MBPP+	Acc.	63.5	69.3	70.6	↑ 1.3	64.8	71.9	72.2	↑ 0.3	68.5	73.0	73.8	↑ 0.8			
	Token	1634	6782	1339	↓ 80.3%	2053	7703	1264	↓ 83.6%	1858	7458	1187	↓ 84.1%			
	Speed	523	1766	489	×3.6	1064	3898	1387	×2.8	2410	9162	2507	×3.7			
HumanEval+	Acc.	75.0	76.2	79.3	↑ 3.1	74.4	76.8	78.0	↑ 1.2	76.8	84.1	86.6	↑ 2.5			
	Token	2380	8127	1373	↓ 83.1%	2507	8768	1274	↓ 85.5%	2366	8114	1512	↓ 81.4%			
	Speed	274	931	333	×2.8	502	1809	439	×4.1	1084	3988	1188	×3.4			

4. Empirical Evaluations

Tasks and Datasets. We conduct a comprehensive evaluation of LatentMAS across nine benchmarks spanning both general-purpose and reasoning-intensive tasks: (i) *Math & Science Reasoning*, including GSM8K (Cobbe et al., 2021), AIME24 (Maxwell-Jia, 2024), AIME25 (math ai, 2025), GPQA-Diamond (Rein et al., 2023), and MedQA (Yang et al., 2024a); (ii) *Commonsense Reasoning*, including ARC-Easy

Table 3 | **Main results of LatentMAS on 3 reasoning-intensive tasks under both Sequential and Hierarchical MAS settings.** We report accuracy, token usage, and end-to-end speed, and highlight the performance gains following the same evaluation protocol as in Table 1.

Tasks	Metrics	Qwen3-8B				Improve	Qwen3-14B			Improve
		Single	TextMAS	LatentMAS	Single		TextMAS	LatentMAS		
Sequential MAS Setting										
AIME24	Acc.	50.0	53.3	56.7	↑ 3.4	63.3	63.3	66.7	↑ 3.4	
	Token	12891	38596	8953	↓ 76.8%	11263	32092	10593	↓ 67.0%	
	Speed	421	2808	688	×4.1	1018	4554	1149	×4.0	
AIME25	Acc.	46.7	53.3	53.3	↑ 0.0	56.7	60.0	63.3	↑ 3.3	
	Token	14692	45088	8699	↓ 80.7%	11298	44618	11402	↓ 74.4%	
	Speed	450	3150	820	×3.8	1040	5184	1473	×3.5	
GPQA-Diamond	Acc.	39.9	43.4	45.5	↑ 2.1	48.5	51.5	52.0	↑ 0.5	
	Token	6435	17986	4571	↓ 74.6%	5547	12676	5454	↓ 57.0%	
	Speed	813	5771	854	×6.8	1043	9714	1475	×6.6	
Hierarchical MAS Setting										
AIME24	Acc.	50.0	53.3	53.3	↑ 0.0	63.3	70.0	73.3	↑ 3.3	
	Token	12891	42629	7526	↓ 82.3%	11263	29025	10230	↓ 64.8%	
	Speed	421	3132	776	×4.0	1018	5718	1089	×5.3	
AIME25	Acc.	46.7	50.0	50.0	↑ 0.0	56.7	66.7	66.7	↑ 0.0	
	Token	14692	53929	13230	↓ 75.5%	11298	50003	9527	↓ 80.9%	
	Speed	450	3488	616	×5.7	1040	6019	1056	×5.7	
GPQA-Diamond	Acc.	39.9	43.0	46.9	↑ 3.9	48.5	52.0	53.0	↑ 1.0	
	Token	6435	22450	3395	↓ 84.9%	5547	20931	3606	↓ 82.8%	
	Speed	813	6108	798	×7.7	1043	9119	1458	×6.3	

(Clark et al., 2018b) and ARC-Challenge (Clark et al., 2018a); and (iii) *Code Generation*, including MBPP-Plus (Liu et al., 2023) and HumanEval-Plus (Liu et al., 2023). Detailed descriptions of each benchmark are provided in Appendix C.1.

Models and Baselines. We adopt three off-the-shelf models from the Qwen3 family (Yang et al., 2025) (4B, 8B, and 14B) to construct LatentMAS at different scales. For baseline comparison, we evaluate LatentMAS against: (i) *Single LLM agents (Single)*, where a single LLM directly performs standard auto-regressive generation with token-level decoding; (ii) *Sequential text-based MAS (Sequential TextMAS)*, following the chain-of-agents design (Zhang et al., 2024b) with text-mediated reasoning and communication; and (iii) *Hierarchical text-based MAS (Hierarchical TextMAS)*, where domain-specialized agents collaborate through a summarizer (Zhuge et al., 2024) using text-based reasoning and communication. Detailed model and baseline implementations are provided in Appendix C.2.

Implementation Details. For latent thoughts generation, we compute the realignment matrix W_a once per run and reuse it across all inference steps. Each LLM agent performs $m \in \{0, 10, 20, 40, 80\}$ latent steps during reasoning. For working memory transfer, we directly concatenate the KV caches from the immediately preceding agent into the corresponding transformer layers through the `past_key_values` interface in HuggingFace Transformers (Face, 2025). Besides the HuggingFace implementation, we also integrate all baseline methods and LatentMAS with the vLLM backend (Kwon et al., 2023), enabling prefix caching and tensor-parallel inference for efficient deployment of larger LLM agents. We perform hyperparameter tuning and report the mean performance over three independent runs. Across both baselines and our method, we set all LLM agents with a temperature of

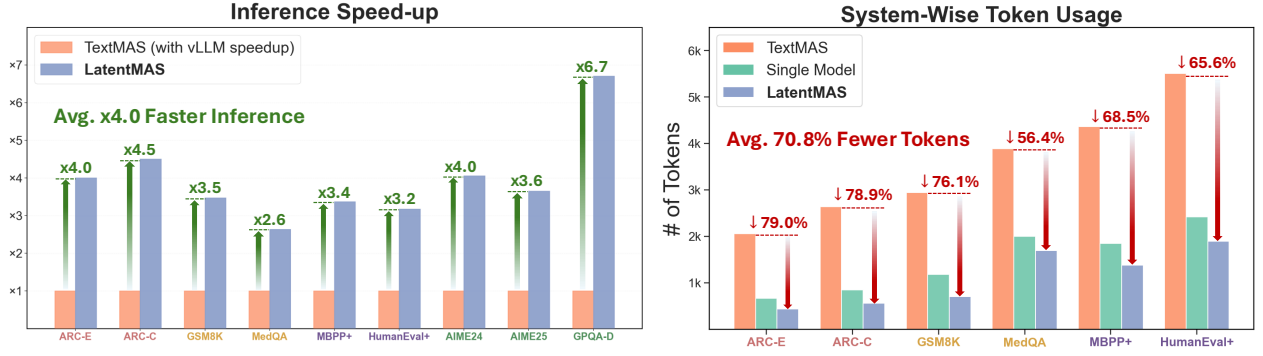


Figure 4 | Efficiency gains of LatentMAS over single model and TextMAS under the sequential MAS setting. **Left:** LatentMAS achieves substantially faster end-to-end inference, even though all baselines are accelerated with vLLM backend. **Right:** LatentMAS requires far fewer system-wise token usage.

0.6 and a top- p of 0.95. We adjust the maximum output length for each task according to its relative difficulty. We set the maximum length to 2,048 tokens for ARC-Easy, ARC-Challenge, and GSM8K, 4096 tokens for MedQA, MBPP+, and HumanEval+, 8,192 tokens for GPQA and 20,000 tokens for AIME24/25. All experiments are conducted on 8×NVIDIA A100-80G GPUs.

4.1. LatentMAS Delivers Higher Accuracy with Efficient Collaboration

Main Results. Tables 1, 2, and 3 report the overall performance of LatentMAS across 9 general and reasoning-intensive benchmarks built from 3 different scales of LLM backbones. To thoroughly examine collaboration behaviors during inference, we evaluate each method from three complementary perspectives: (i) *task accuracy*, (ii) *system throughput (total output tokens)*, and (iii) *end-to-end inference speed*. Across all tasks, LatentMAS consistently improves over the single-model baseline by an average of 14.6% and 13.3% under the sequential and hierarchical settings, respectively, and further yields gains of 2.8% and 4.6% over text-based MAS. Under identical MAS architectures, LatentMAS provides 4× and 4.3× faster inference speed on average compared with sequential and hierarchical text-based MAS. Additionally, as the entire collaboration occurs entirely in latent space, LatentMAS reduces token usage significantly by 70.8% and 83.7% relative to sequential and hierarchical TextMAS.

Superior Efficiency on Latent Collaboration. As early established in Theorem 3.1, LatentMAS can theoretically achieve orders-of-magnitude higher efficiency than text-based MAS. We further empirically validate this advantage through efficiency analyses comparing LatentMAS with TextMAS. As visualized in Figure 1 and 4 (left), even after accelerating the TextMAS baselines using the vLLM service, LatentMAS still achieves a 2.6×-7× speedup over the vLLM-optimized TextMAS. This improvement stems from the substantially reduced number of latent steps required for latent thoughts generation compared with the much larger decoding steps needed for per-token text generation. For instance, with fewer than 50 latent steps, LatentMAS attains comparable or even higher performance on reasoning-intensive tasks such as AIME 24/25, whereas TextMAS typically requires more than 20K output tokens to complete full text-based CoT trajectories.

In addition, as illustrated in Figure 1 and 4 (right), LatentMAS reduces token usage by 59.4%-87.9% compared with TextMAS, as agents in LatentMAS communicate by directly transferring latent working memory into another agent’s internal layers rather than relying on a text-based medium. Notably, LatentMAS also achieves 15.0%-60.3% lower token usage than single agents. Compared with single-model reasoning, LatentMAS distributes the input question across multiple collaborating agents, greatly reducing the burden on the final agent, which primarily aggregates preceding latent thoughts

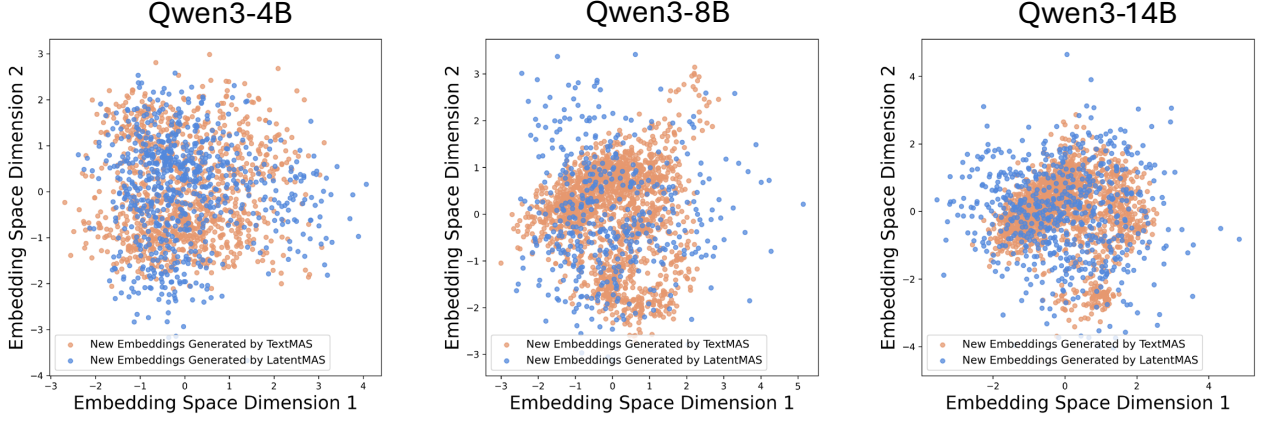


Figure 5 | **Illustration of the semantic meaning encoded by latent thoughts in LatentMAS.** Newly generated latent thought embeddings in LatentMAS largely cover the embedding space of text-based generated tokens, indicating semantic consistency and greater expressive capacity than discrete text.

and decodes the final answer using only a small number of tokens. As a result, the entire system generates fewer output tokens while still achieving higher accuracy.

4.2. In-depth Analyses on LatentMAS

Do Latent Thoughts Reflect Text Reasoning? We first verify whether latent thoughts generation in LatentMAS produces meaningful and semantically expressive representations. To this end, we compare the distribution of newly generated last-layer embeddings in LatentMAS with the embeddings of token-by-token responses produced by TextMAS. Experiments are conducted on 300 MedQA questions, using 40 latent steps for LatentMAS and a 4096 max-token budget for the TextMAS baseline.

As shown in Figure 5, we highlight two key observations: (i) The last-layer embeddings from LatentMAS share nearly the same region of the embedding space with the token embeddings from TextMAS, indicating that latent thoughts encode similar semantic representations as the correct text responses. (ii) The last-layer embeddings from LatentMAS largely cover the distribution of token embeddings from TextMAS, indicating that latent thoughts offer greater diversity and expressive capacity than discrete tokens. Together, these findings show that latent thoughts not only capture the valid semantics of their corresponding text responses but also encode richer and more expressive representations inside. We further include a case study in Appendix D analyzing how LLM agents in LatentMAS interpret their own latent thoughts to provide additional validation for our claim.

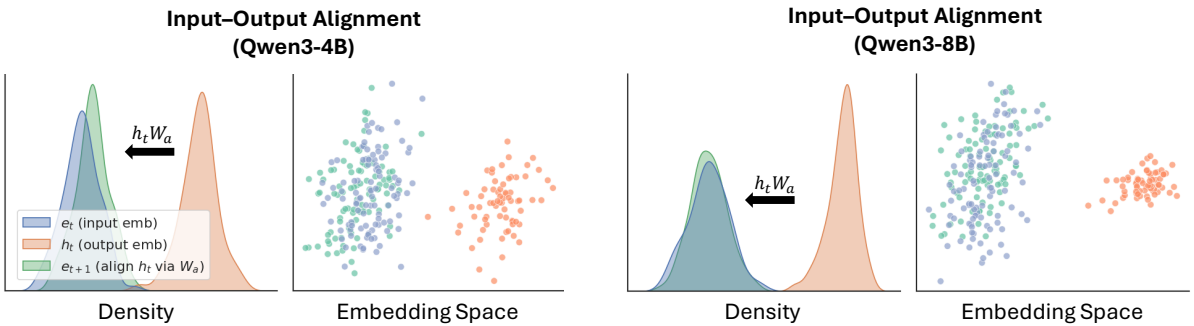


Figure 6 | **Effectiveness of the input-output alignment W_a on MedQA.** Unaligned output embeddings (h_t) drift away from the original input embeddings (e_t), while the aligned vectors (e_{t+1}) realign with e_t , demonstrating that W_a preserves embedding-space structure and prevents representation drift.

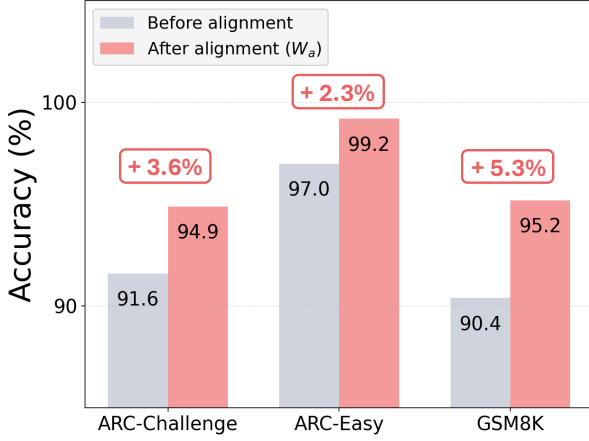


Figure 7 | Downstream performance before/after applying the input-output alignment W_a .

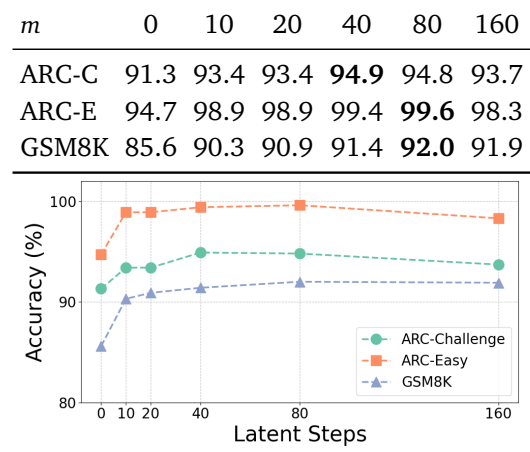


Figure 8 | Effectiveness of different latent step depths of LatentMAS on downstream performance.

Effectiveness on Input-Output Alignment. We next empirically evaluate the effectiveness of the input-output alignment in our method design. First, we compare the input vector e_t obtained from the standard token embedding layer with both the newly generated output vector h_t before alignment and the after-aligned vector e_{t+1} . As shown in Figure 6, we visualize the three embedding vectors by comparing both density distributions and geometric relationships in the projected embedding space. We observe that the new h_t deviates largely from the original input embedding e_t . After applying W_a , the aligned vector e_{t+1} realigns with e_t , indicating that W_a effectively restores the geometric and statistical structure of the input embedding space and mitigates representation drift across iterative latent steps. In Figure 7, we further compare downstream performance before and after applying W_a and observe consistent accuracy gains of 2.3%-5.3% brought by W_a .

Optimal Latent Step Depth. To understand how many latent steps are needed for optimal performance in LatentMAS, we analyze the effect of increasing latent step depth across three downstream tasks. As shown in Figure 8, increasing the number of latent steps generally improves downstream performance, indicating that additional latent thoughts enhance collaborative expressiveness. Across the three tasks on Qwen3-14B, we find that accuracy steadily rises and peaks around 40-80 steps. Beyond this range, performance plateaus or declines, suggesting that excessive latent thought generation may introduce redundant or less useful information. Based on this observation, we adopt a moderate latent step budget within this range in practice, as it consistently provides the best accuracy-efficiency trade-off without requiring any task-specific training procedures.

5. Related Work

LLM-based Multi-agent Systems. Recent studies in Agentic AI have extended classical multi-agent systems (Park et al., 2023a; Yang et al., 2024b) grounded in traditional reinforcement learning and policy coordination (Li et al., 2025b; Tan et al., 2025), to modern LLM settings, enabling models to operate as autonomous agents that collaborate in reasoning, planning, and problem-solving (Tao et al., 2024; Wang et al., 2025b; Zhao et al., 2025b). Early investigations, such as ReAct (Yao et al., 2022), AutoGen (Wu et al., 2024), and CAMEL (Li et al., 2023), coordinate multiple LLMs through explicit dialogue or role assignment to improve task diversity and reliability. Additional methods introduce structured communication protocols or training paradigms to enhance cooperation efficiency (Chen et al., 2025a; Yan et al., 2025; Ye et al., 2025a; Zou et al., 2025) and emergent specialization (Huang et al., 2025; Mieczkowski et al., 2025) among agents. In summary, a large amount of prior works follow

sequential planner-solver pipelines or hierarchical expert-summarizer structures, which correspond to the two MAS settings we adopt for evaluating LatentMAS. Beyond algorithmic advances, LLM-based MAS have been applied across diverse domains, such as math and science reasoning (Pezeshkpour et al., 2024; Yue et al., 2024), open-domain question answering (Fourney et al., 2024; Wu et al., 2025), and multi-modal GUI interaction (Ye et al., 2025b; Zhang et al., 2024a), demonstrating their versatility in complex real-world settings. Building upon these advanced text-MAS methods, our work aims to enable a latent-based multi-agent collaboration system, treating agents as tightly integrated components that achieve more efficient coordination and expressive capabilities.

Model Collaboration in Latent Space. Recent works on model ensemble collaboration (Sagi and Rokach, 2018; Zhang and Ma, 2012) have shifted from text-level coordination to latent-space interaction. For example, ThoughtComm (Zheng et al., 2025) learns a shared latent space and routes agent information through trained encoder-decoder and prefix modules on top of frozen LLMs. Cache-to-Cache (Fu et al., 2025) enables semantic transfer across two models by projecting the sharer model’s input-prompt’s KV into a receiver model. Mixture of Thoughts (Fein-Ashley et al., 2025) leverages a primary expert model to aggregate cross-attention information of other models, enabling single-pass, centrally controlled latent fusion. On the other hand, LatentMAS is a training-free latent MAS in which each model first generates its own latent thoughts, and then the input with the newly generated thoughts is transferred from one model to another for subsequent collaboration.

Latent Reasoning in LLMs. Beyond explicit chain-of-thought (CoT) reasoning, recent work has explored the continuous latent space of LLMs as an alternative reasoning medium (Chen et al., 2025b; Hao et al., 2024), revealing that hidden states encode richer semantic structures than what discrete token generation can express (Liu et al., 2024; Zhang et al., 2025). Latent reasoning methods such as CoCoNut (Hao et al., 2024) and latent-space editing approaches (e.g., RepE (Zou et al., 2023), LoT (Fungwacharakorn et al., 2024)) demonstrate that manipulating internal representations can guide models to reason more coherently and improve controllability without explicit token-level rationales. Other works (Fein-Ashley and Fein-Ashley, 2025; Li et al., 2025a; Wang et al., 2025a) have also extended latent reasoning paradigms to vision-language models. These methods leverage the structure of hidden states to perform interventions, such as steering, editing, or optimizing latent trajectories, that shape downstream reasoning behavior while remaining agnostic to surface-level text. By operating directly in the continuous space, they can induce reasoning steps that would be difficult or inefficient to express (Coda-Forno et al., 2025; Liu et al., 2024; Zhang et al., 2025). Despite these benefits, existing techniques are confined to a single model’s internal computations and do not consider interaction or coordination across multiple reasoning entities (Hao et al., 2024). On the other hand, LatentMAS extends latent reasoning to a multi-agent setting, enabling each agent to generate latent thoughts and propagate latent information to others. Our new framework shifts latent reasoning from an isolated capability of individual models to a system-level collaborative mechanism.

6. Conclusion

We introduced LatentMAS, a training-free framework that enables multi-agent systems to collaborate entirely within the continuous latent space. By combining latent auto-regressive reasoning with a lossless latent working-memory transfer mechanism, LatentMAS overcomes the inherent inefficiencies and information bottlenecks of text-based collaboration. Our theoretical analyses establish substantial gains in expressiveness and computational efficiency, and our empirical results across diverse reasoning, commonsense, and code-generation benchmarks demonstrate that latent collaboration consistently improves accuracy performance, token usage, and decoding speed over strong single-model and text-based MAS baselines. Together, LatentMAS serves as a scalable and general paradigm for building next-generation agentic systems that cooperate beyond the limits of natural language. An exciting

future direction is to adapt advanced post-training paradigms from text-based MAS to optimize LatentMAS's latent collaboration protocols to unlock more effective multi-agent reasoning strategies.

References

- D. B. Acharya, K. Kuppan, and B. Divya. Agentic ai: Autonomous intelligence for complex goals—a comprehensive survey. *IEEE Access*, 2025.
- S. K. Ainsworth, J. Hayase, and S. Srinivasa. Git re-basin: Merging models modulo permutation symmetries. *arXiv preprint arXiv:2209.04836*, 2022.
- M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, et al. Why do multi-agent llm systems fail? *arXiv preprint arXiv:2503.13657*, 2025.
- W. Chen, J. Yuan, C. Qian, C. Yang, Z. Liu, and M. Sun. Optima: Optimizing effectiveness and efficiency for llm-based multi-agent system. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 11534–11557, 2025a.
- X. Chen, A. Zhao, H. Xia, X. Lu, H. Wang, Y. Chen, W. Zhang, J. Wang, W. Li, and X. Shen. Reasoning beyond language: A comprehensive survey on latent chain-of-thought reasoning. *arXiv preprint arXiv:2505.16782*, 2025b.
- P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018a.
- P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018b.
- K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- J. Coda-Forno, Z. Zhao, Q. Zhang, D. Tamboli, W. Li, X. Fan, L. Zhang, E. Schulz, and H.-P. Tseng. Exploring system 1 and 2 communication for latent reasoning in llms. *arXiv preprint arXiv:2510.00494*, 2025.
- H. Face. Transformers documentation. <https://huggingface.co/docs/transformers/en/index>, 2025.
- B. Fein-Ashley and J. Fein-Ashley. Bridging hidden states in vision-language models. *arXiv preprint arXiv:2511.11526*, 2025.
- J. Fein-Ashley, D. Parikh, R. Kannan, and V. Prasanna. Mixture of thoughts: Learning to aggregate what experts think, not just what they say. *arXiv preprint arXiv:2509.21164*, 2025.
- Z. Feng, R. Xue, L. Yuan, Y. Yu, N. Ding, M. Liu, B. Gao, J. Sun, X. Zheng, and G. Wang. Multi-agent embodied ai: Advances and future directions. *arXiv preprint arXiv:2505.05108*, 2025.
- A. Fourney, G. Bansal, H. Mozannar, C. Tan, E. Salinas, F. Niedtner, G. Proebsting, G. Bassman, J. Gerrits, J. Alber, et al. Magentic-one: A generalist multi-agent system for solving complex tasks. *arXiv preprint arXiv:2411.04468*, 2024.

- T. Fu, Z. Min, H. Zhang, J. Yan, G. Dai, W. Ouyang, and Y. Wang. Cache-to-cache: Direct semantic communication between large language models. *arXiv preprint arXiv:2510.03215*, 2025.
- W. Fungwacharakorn, N. H. Thanh, M. M. Zin, and K. Satoh. Layer-of-thoughts prompting (lot): Leveraging llm-based retrieval with constraint hierarchies. *arXiv preprint arXiv:2410.12153*, 2024.
- T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024.
- S. Hao, S. Sukhbaatar, D. Su, X. Li, Z. Hu, J. Weston, and Y. Tian. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.
- A. E. Hoerl and R. W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.
- S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2023.
- M. Hu, Y. Zhou, W. Fan, Y. Nie, B. Xia, T. Sun, Z. Ye, Z. Jin, Y. Li, Q. Chen, et al. Owl: Optimized workforce learning for general multi-agent assistance in real-world task automation. *arXiv preprint arXiv:2505.23885*, 2025.
- Q. Huang, Z. Zhou, Y. Li, K. Yang, B. Wang, and Y. Wang. Many minds, one goal: Time series forecasting via sub-task specialization and inter-agent cooperation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- B. Jin, H. Zeng, Z. Yue, J. Yoon, S. Arik, D. Wang, H. Zamani, and J. Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.
- W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.
- B. Li, X. Sun, J. Liu, Z. Wang, J. Wu, X. Yu, H. Chen, E. Barsoum, M. Chen, and Z. Liu. Latent visual reasoning. *arXiv preprint arXiv:2509.24251*, 2025a.
- G. Li, H. A. Al Kader Hammoud, H. Itani, D. Khizbullin, and B. Ghanem. Camel: communicative agents for "mind" exploration of large language model society. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23*, Red Hook, NY, USA, 2023. Curran Associates Inc.
- Z. Li, Q. Ji, X. Ling, and Q. Liu. A comprehensive review of multi-agent reinforcement learning in video games. *IEEE Transactions on Games*, 2025b.
- Z. Li, W. Wu, Y. Guo, J. Sun, and Q.-L. Han. Embodied multi-agent systems: A review. *IEEE/CAA Journal of Automatica Sinica*, 12(6):1095–1116, 2025c.
- Z. Li, H. Zhang, S. Han, S. Liu, J. Xie, Y. Zhang, Y. Choi, J. Zou, and P. Lu. In-the-flow agentic system optimization for effective planning and tool use. *arXiv preprint arXiv:2510.05592*, 2025d.
- J. Liu, C. S. Xia, Y. Wang, and L. Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems*, 36:21558–21572, 2023.

- L. Liu, J. Pfeiffer, J. Wu, J. Xie, and A. Szlam. Deliberation in latent space via differentiable cache augmentation. *arXiv preprint arXiv:2412.17747*, 2024.
- math ai. AIME 2025 dataset. <https://huggingface.co/datasets/math-ai/aime25>, 2025.
- Maxwell-Jia. AIME 2024 dataset. https://huggingface.co/datasets/Maxwell-Jia/AIME_2024, 2024.
- L. Meegahapola, V. Subramaniam, L. Kaplan, and A. Misra. Prior activation distribution (pad): A versatile representation to utilize dnn hidden units. *arXiv preprint arXiv:1907.02711*, 2019.
- E. Mieczkowski, R. Mon-Williams, N. Bramley, C. G. Lucas, N. Velez, and T. L. Griffiths. Predicting multi-agent specialization via task parallelizability. *arXiv preprint arXiv:2503.15703*, 2025.
- J. S. Park, J. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023a.
- K. Park, Y. J. Choe, and V. Veitch. The linear representation hypothesis and the geometry of large language models. *arXiv preprint arXiv:2311.03658*, 2023b.
- P. Pezeshkpour, E. Kandogan, N. Bhutani, S. Rahman, T. Mitchell, and E. Hruschka. Reasoning capacity in multi-agent systems: Limitations, challenges and human-centered solutions. *arXiv preprint arXiv:2402.01108*, 2024.
- D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark, 2023. URL <https://arxiv.org/abs/2311.12022>.
- O. Sagi and L. Rokach. Ensemble learning: A survey. *Wiley interdisciplinary reviews: data mining and knowledge discovery*, 8(4):e1249, 2018.
- L. Tan, F. Wei, X. Ma, R. Peng, H. Xiao, and L. Yang. Systemic condition-based maintenance optimization under inspection uncertainties: A customized multiagent reinforcement learning approach. *IEEE Transactions on Reliability*, 2025.
- W. Tao, Y. Zhou, Y. Wang, W. Zhang, H. Zhang, and Y. Cheng. Magis: Llm-based multi-agent framework for github issue resolution. *Advances in Neural Information Processing Systems*, 37:51963–51993, 2024.
- K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O’Sullivan, and H. D. Nguyen. Multi-agent collaboration mechanisms: A survey of llms. *arXiv preprint arXiv:2501.06322*, 2025.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin. Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- Q. Wang, Y. Shi, Y. Wang, Y. Zhang, P. Wan, K. Gai, X. Ying, and Y. Wang. Monet: Reasoning in latent visual space beyond images and language. *arXiv preprint arXiv:2511.21395*, 2025a.
- Z. Wang, S. Moriyama, W.-Y. Wang, B. Gangopadhyay, and S. Takamatsu. Talk structurally, act hierarchically: A collaborative framework for llm multi-agent systems. *arXiv preprint arXiv:2502.11098*, 2025b.

- M. Wortsman, G. Ilharco, S. Y. Gadre, R. Roelofs, R. Gontijo-Lopes, A. S. Morcos, H. Namkoong, A. Farhadi, Y. Carmon, S. Kornblith, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International conference on machine learning*, pages 23965–23998. PMLR, 2022.
- F. Wu, Z. Li, F. Wei, Y. Li, B. Ding, and J. Gao. Talk to right specialists: Routing and planning in multi-agent system for question answering. *arXiv preprint arXiv:2501.07813*, 2025.
- Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.
- B. Yan, Z. Zhou, L. Zhang, L. Zhang, Z. Zhou, D. Miao, Z. Li, C. Li, and X. Zhang. Beyond self-talk: A communication-centric survey of llm-based multi-agent systems. *arXiv preprint arXiv:2502.14321*, 2025.
- A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- H. Yang, H. Chen, H. Guo, Y. Chen, C.-S. Lin, S. Hu, J. Hu, X. Wu, and X. Wang. Llm-medqa: Enhancing medical question answering through case studies in large language models. *arXiv preprint arXiv:2501.05464*, 2024a.
- Y. Yang, Q. Peng, J. Wang, Y. Wen, and W. Zhang. Llm-based multi-agent systems: Techniques and business perspectives. *arXiv preprint arXiv:2411.14033*, 2024b.
- S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- H. Ye, Z. Gao, M. Ma, Q. Wang, Y. Fu, M.-Y. Chung, Y. Lin, Z. Liu, J. Zhang, D. Zhuo, et al. Kvcomm: Online cross-context kv-cache communication for efficient llm-based multi-agent systems. *arXiv preprint arXiv:2510.12872*, 2025a.
- J. Ye, X. Zhang, H. Xu, H. Liu, J. Wang, Z. Zhu, Z. Zheng, F. Gao, J. Cao, Z. Lu, et al. Mobile-agent-v3: Fundamental agents for gui automation. *arXiv preprint arXiv:2508.15144*, 2025b.
- L. Yue, S. Xing, J. Chen, and T. Fu. Clinicalagent: Clinical trial multi-agent system with large language model-based reasoning. In *Proceedings of the 15th ACM International Conference on Bioinformatics, Computational Biology and Health Informatics*, pages 1–10, 2024.
- C. Zhang and Y. Ma. *Ensemble machine learning*, volume 144. Springer, 2012.
- C. Zhang, S. He, J. Qian, B. Li, L. Li, S. Qin, Y. Kang, M. Ma, G. Liu, Q. Lin, et al. Large language model-brained gui agents: A survey. *arXiv preprint arXiv:2411.18279*, 2024a.
- Y. Zhang, R. Sun, Y. Chen, T. Pfister, R. Zhang, and S. Arik. Chain of agents: Large language models collaborating on long-context tasks. *Advances in Neural Information Processing Systems*, 37: 132208–132237, 2024b.
- Z. Zhang, X. He, W. Yan, A. Shen, C. Zhao, S. Wang, Y. Shen, and X. E. Wang. Soft thinking: Unlocking the reasoning potential of llms in continuous concept space. *arXiv preprint arXiv:2505.15778*, 2025.
- J. Zhao, H. Xie, Y. Lei, X. Song, Z. Shi, L. Li, S. Liu, and H. Zhang. Connecting the dots: A chain-of-collaboration prompting framework for llm agents. *arXiv preprint arXiv:2505.10936*, 2025a.

- W. Zhao, M. Yuksekgonul, S. Wu, and J. Zou. Sirius: Self-improving multi-agent systems via bootstrapped reasoning. *arXiv preprint arXiv:2502.04780*, 2025b.
- Y. Zheng, Z. Zhao, Z. Li, Y. Xie, M. Gao, L. Zhang, and K. Zhang. Thought communication in multiagent collaboration. *arXiv preprint arXiv:2510.20733*, 2025.
- H. Zhou, H. Geng, X. Xue, L. Kang, Y. Qin, Z. Wang, Z. Yin, and L. Bai. Reso: A reward-driven self-organizing llm-based multi-agent system for reasoning tasks. *arXiv preprint arXiv:2503.02390*, 2025.
- W. Zhou, J. Du, and X. Ren. Improving bert fine-tuning with embedding normalization. *arXiv preprint arXiv:1911.03918*, 2019.
- H. Zhu, S. Hao, Z. Hu, J. Jiao, S. Russell, and Y. Tian. Reasoning by superposition: A theoretical perspective on chain of continuous thought. *arXiv preprint arXiv:2505.12514*, 2025.
- M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, and J. Schmidhuber. Language agents as optimizable graphs. *arXiv preprint arXiv:2402.16823*, 2024.
- A. Zou, L. Phan, S. Chen, J. Campbell, P. Guo, R. Ren, A. Pan, X. Yin, M. Mazeika, A.-K. Dombrowski, et al. Representation engineering: A top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405*, 2023.
- J. Zou, Y. Ban, Z. Li, Y. Qi, R. Qiu, L. Yang, and J. He. Transformer copilot: Learning from the mistake log in LLM fine-tuning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

Table of Contents

A	Input-Output Alignment in LatentMAS	19
A.1	Solving the Alignment Matrix W_a	19
A.2	Theoretical Justification on W_a	19
B	Theoretical Analysis	22
B.1	Proof of Theorem 3.1	22
B.2	Proof of Theorem 3.3	22
B.3	Proof of Theorem 3.4	23
C	Experiment Setups	25
C.1	Evaluation Details	25
C.2	Implementation Details	26
C.3	Additional Discussions on LatentMAS	26
D	Case Study	27
E	Prompt Template for LatentMAS	29

Appendix

A. Input-Output Alignment in LatentMAS

A.1. Solving the Alignment Matrix W_a

In Section 3.1, we put the last-layer hidden states h back to the input sequence to enable the model’s latent reasoning. However, since the h is not perfectly aligned with the input embedding space, directly feeding h into shallow layers may lead to out-of-distribution activation patterns inside LLMs. To mitigate this in a training-free way, we seek a matrix W_a which maps h to a valid input space (i.e., $e = hW_a$). A straightforward way to calculate W_a is to enforce that the aligned latent vector e behaves similarly to a real input embedding when it enters the model. Motivated by our Theorem A.1 below, this corresponds to the following minimization problem:

$$\min_{W_a} \|W_{\text{out}}W_a - W_{\text{in}}\|_F^2. \quad (5)$$

This objective is quadratic in W_a , so we can derive a closed-form solution by setting its derivative to zero, which yields the normal equation:

$$W_{\text{out}}^\top W_{\text{out}}W_a - W_{\text{out}}^\top W_{\text{in}} = 0. \quad (6)$$

Solving for W_a gives:

$$W_a = (W_{\text{out}}^\top W_{\text{out}})^{-1} W_{\text{out}}^\top W_{\text{in}}. \quad (7)$$

For numerical stability, we further add a small hyperparameter $\lambda > 0$ to obtain a ridge regression solution:

$$W_a = (W_{\text{out}}^\top W_{\text{out}} + \lambda I)^{-1} W_{\text{out}}^\top W_{\text{in}}, \quad (8)$$

which we compute once and reuse for all latent reasoning steps.

A.2. Theoretical Justification on W_a

In this section, we outline the theoretical justification for how W_a minimizes the distributional gap between the distribution of token embeddings and the distribution of aligned embeddings.

Let P_e and P_h be the distribution of token embeddings e and the hidden embeddings h , respectively. We assume that P_e and P_h can be generated by $e = W_{\text{in},x}$ and $h = W_{\text{out},x}$, respectively, where token x follows an underlying token distribution $x \sim P_{\mathcal{V}}$. For an alignment matrix W_a , the aligned embedding distribution $P_{\hat{e},W_a}$ is

$$P_{\hat{e},W_a} : \hat{e} = hW_a, \quad h \sim P_h. \quad (9)$$

Our goal is to minimize the distance between the aligned embedding distribution $P_{\hat{e},W_a}$ and the token embedding distribution P_e , which we measure via the Wasserstein distance:

$$d_{\text{Wasserstein}}(P_{\hat{e},W_a}, P_e) := \inf_{\gamma \in \Gamma(P_e, P_{\hat{e},W_a})} \sqrt{\mathbb{E}_{(\hat{e},e) \sim \gamma} [\|\hat{e} - e\|_2^2]}, \quad (10)$$

where $\Gamma(P_{\hat{e},W_a}, P_e)$ is the set of all couplings of P_e and $P_{\hat{e},W_a}$.

Theorem A.1 (Upper Bound on Distribution Alignment). *Suppose that the rows of W_{in} and W_{out} are mutually distinct. Then for any non-singular alignment matrix W_a , the Wasserstein distance between P_e and $P_{\hat{e}, W_a}$ is upper bounded by*

$$d_{\text{Wasserstein}}(P_{\hat{e}, W_a}, P_e) \leq \|W_{\text{out}}W_a - W_{\text{in}}\|_F. \quad (11)$$

As we show in Appendix A.1, our choice of W_a (Equation 3) minimizes this upper bound of $W(P_{\hat{e}, W_a}, P_e)$.

Proof. Consider the following joint distribution $\gamma^*(\hat{e}, e)$:

$$\gamma^*(\hat{e}, e) := \sum_{x \in \mathcal{V}} P_{\mathcal{V}}(x) \mathbf{1}_{[W_{\text{out}, x}W_a = \hat{e}]} \mathbf{1}_{[W_{\text{in}, x} = e]}. \quad (12)$$

Since the rows of W_{in} are mutually distinct, then for every $\hat{e}$,

$$\sum_{e \in \text{supp}(P_e)} \gamma^*(\hat{e}, e) = \sum_{e \in \text{supp}(P_e)} \sum_{x \in \mathcal{V}} P_{\mathcal{V}}(x) \mathbf{1}_{[W_{\text{out}, x}W_a = \hat{e}]} \mathbf{1}_{[W_{\text{in}, x} = e]} \quad (13)$$

$$= \sum_{x \in \mathcal{V}} P_{\mathcal{V}}(x) \mathbf{1}_{[W_{\text{out}, x}W_a = \hat{e}]} \sum_{e \in \text{supp}(P_e)} \mathbf{1}_{[W_{\text{in}, x} = e]} \quad (14)$$

$$= \sum_{x \in \mathcal{V}} P_{\mathcal{V}}(x) \mathbf{1}_{[W_{\text{out}, x}W_a = \hat{e}]} \quad (15)$$

$$= P_{\hat{e}, W_a}(\hat{e}); \quad (16)$$

and since the rows of W_{out} are mutually distinct, and W_a is non-singular, then for every e ,

$$\sum_{\hat{e} \in \text{supp}(P_{\hat{e}, W_a})} \gamma^*(\hat{e}, e) = \sum_{\hat{e} \in \text{supp}(P_{\hat{e}, W_a})} \sum_{x \in \mathcal{V}} P_{\mathcal{V}}(x) \mathbf{1}_{[W_{\text{out}, x}W_a = \hat{e}]} \mathbf{1}_{[W_{\text{in}, x} = e]} \quad (17)$$

$$= \sum_{x \in \mathcal{V}} P_{\mathcal{V}}(x) \mathbf{1}_{[W_{\text{in}, x} = e]} \sum_{\hat{e} \in \text{supp}(P_{\hat{e}, W_a})} \mathbf{1}_{[W_{\text{out}, x}W_a = \hat{e}]} \quad (18)$$

$$= \sum_{x \in \mathcal{V}} P_{\mathcal{V}}(x) \mathbf{1}_{[W_{\text{in}, x} = e]} \quad (19)$$

$$= P_e(e). \quad (20)$$

This implies $\gamma^* \in \Gamma(P_{\hat{e}, W_a}, P_e)$. It follows that

$$d_{\text{Wasserstein}}(P_{\hat{e}, W_a}, P_e) = \inf_{\gamma \in \Gamma(P_e, P_{\hat{e}, W_a})} \sqrt{\mathbb{E}_{(\hat{e}, e) \sim \gamma} [\|\hat{e} - e\|_2^2]} \quad (21)$$

$$\leq \sqrt{\mathbb{E}_{(\hat{e}, e) \sim \gamma^*} [\|\hat{e} - e\|_2^2]} \quad (22)$$

$$= \sqrt{\sum_{\hat{e} \in \text{supp}(P_{\hat{e}, W_a})} \sum_{e \in \text{supp}(P_e)} \gamma^*(\hat{e}, e) \|\hat{e} - e\|_2^2} \quad (23)$$

$$= \sqrt{\sum_{\hat{e} \in \text{supp}(P_{\hat{e}, W_a})} \sum_{e \in \text{supp}(P_e)} \sum_{x \in \mathcal{V}} P_{\gamma}(x) 1_{[W_{\text{out}, x} W_a = \hat{e}]} 1_{[W_{\text{in}, x} = e]} \|\hat{e} - e\|_2^2} \quad (24)$$

$$= \sqrt{\sum_{x \in \mathcal{V}} P_{\gamma}(x) \sum_{\hat{e} \in \text{supp}(P_{\hat{e}, W_a})} \sum_{e \in \text{supp}(P_e)} 1_{[W_{\text{out}, x} W_a = \hat{e}]} 1_{[W_{\text{in}, x} = e]} \|\hat{e} - e\|_2^2} \quad (25)$$

$$= \sqrt{\sum_{x \in \mathcal{V}} P_{\gamma}(x) \|W_{\text{out}, x} W_a - W_{\text{in}, x}\|_2^2} \quad (26)$$

$$\leq \sqrt{\sum_{x \in \mathcal{V}} \|W_{\text{out}, x} W_a - W_{\text{in}, x}\|_2^2} \quad (27)$$

$$= \sqrt{\|W_{\text{out}} W_a - W_{\text{in}}\|_F^2} \quad (28)$$

$$= \|W_{\text{out}} W_a - W_{\text{in}}\|_F. \quad (29)$$

□

B. Theoretical Analysis

B.1. Proof of Theorem 3.1

Assumption B.1 (Linear Representation Hypothesis; Park et al., 2023b). We assume that the hidden embeddings h are linear combinations $\sum_{i=1}^{d_h} c_i s_i$ of an underlying semantic basis $\{s_1, \dots, s_{d_h}\} \subset \mathbb{R}^{d_h}$ (linearly independent) with ternary coefficients $c_1, \dots, c_{d_h} \in \{0, \pm 1\}$, where $c_i = 0$ represents that h does not have semantic i , and $c_i = \pm 1$ represents that h has semantic i in a positive/negative way.

Theorem B.1 (Restate of Theorem 3.1). Under the Linear Representation Hypothesis on h , if the sequence of all latent thoughts with length m can be expressed losslessly through corresponding text-based reasoning, then the length of text (in tokens) needs to be at least $\Omega(d_h m / \log |\mathcal{V}|)$, where $|\mathcal{V}| > 1$ denotes the vocabulary size.

Proof of Theorem 3.1. Under Assumption B.1, the set $\mathcal{H}$ of hidden embeddings is

$$\mathcal{H} = \left\{ \sum_{i=1}^{d_h} c_i s_i : c_1, \dots, c_{d_h} \in \{0, \pm 1\} \right\}, \quad (30)$$

where $\{s_1, \dots, s_{d_h}\} \subset \mathbb{R}^{d_h}$ is the underlying semantic basis. Then, the set of length- t latent reasoning sequences is $\mathcal{H}^m$. Since the semantic basis is linearly independent, the size of the set $\mathcal{H}$ of hidden embeddings is

$$|\mathcal{H}| = |\{0, \pm 1\}|^{|\{s_1, \dots, s_{d_h}\}|} = 3^{d_h}. \quad (31)$$

Thus, the size of the set of length- m latent reasoning sequences is

$$|\mathcal{H}^m| = |\mathcal{H}|^m = (3^{d_h})^m = 3^{d_h m}. \quad (32)$$

To represent the set $\mathcal{H}^m$ of length- m latent reasoning sequences via the set $\mathcal{V}^{m'}$ of length- m' text-based reasoning sequences losslessly, there needs to exist an surjective map from $\mathcal{V}^{m'}$ to $\mathcal{H}^m$, which implies that $|\mathcal{V}^{m'}| \geq |\mathcal{H}^m|$. Therefore,

$$m' = \log_{|\mathcal{V}|}(|\mathcal{V}|^{m'}) = \log_{|\mathcal{V}|} |\mathcal{V}^{m'}| \quad (33)$$

$$\geq \log_{|\mathcal{V}|} |\mathcal{H}^m| = \log_{|\mathcal{V}|} (3^{d_h m}) \quad (34)$$

$$= \frac{d_h m \log 3}{\log |\mathcal{V}|} = \Omega\left(\frac{d_h m}{\log |\mathcal{V}|}\right). \square \quad (35)$$

B.2. Proof of Theorem 3.3

Theorem B.2 (Restate of Theorem 3.3). In both latent and text-based reasoning, the outputs of an agent when receiving latent working memory from preceding agents are equivalent to those obtained when directly inputting the preceding agents' outputs.

Proof. Let $h^{(l)}, K^{(l)}, V^{(l)}$ and $h'^{(l)}, K'^{(l)}, V'^{(l)}$ denote the output, keys, and values of l -th transformer layer when receiving latent working memory from preceding agents and when directly inputting the preceding agents' outputs, respectively. In the following, we will use induction to show that $h^{(l)} = h'^{(l)}$ for every layer $l = 1, \dots, L$.

Induction step. Suppose that $h^{(l-1)} = h'^{(l-1)}$, and we will show that $h^{(l)} = h'^{(l)}$.

The KV cache contains $K_{\leq t+m}^{(l)}$ and $V_{\leq t+m}^{(l)}$. For each past token layer, at each attention layer, the transformer produces one column of $K_{\leq t+m}^{(l)}$ and a corresponding column of $V_{\leq t+m}^{(l)}$. At the next step the model forms a query from the current input and then uses that query together with the stored $K_{\leq t+m}^{(l)}$ and $V_{\leq t+m}^{(l)}$ to form the attention result. That attention result is a deterministic function of the query and of the keys and values it attends to.

We are comparing two ways to make those same keys and values available to the current computation: (i) actually feeding the earlier tokens into the model again, in which case the model will recompute the same keys and values and then use them in attention; (ii) reading in $K_{\leq t+m}^{(l)}$ and $V_{\leq t+m}^{(l)}$ from the cache and use them directly. In both cases, the keys and values presented to the attention computation are identical, because the cache was produced by the same model on the same inputs.

Given identical keys and values and the same current input, the attention output is the same in both scenarios. The remainder of the transformer computation that produces the last-layer hidden embedding is a deterministic function of that attention output (and the current input). Therefore, the last-layer hidden embedding $h^{(l)}$ produced for the current step is the same whether the model recomputed keys/values from tokens or read $K_{\leq t+m}^{(l)}$, $V_{\leq t+m}^{(l)}$ from cache. Formally, since $h^{(l-1)} = h'^{(l-1)}$, $K_{\leq t+m}^{(l)} = K'_{\leq t+m}^{(l)}$, and $V_{\leq t+m}^{(l)} = V'_{\leq t+m}^{(l)}$, then $h^{(l)} = h'^{(l)}$.

Induction base case. For the first layer, similarly with the induction step, since the input is the same (for both latent-based and text-based reasoning), $K_{\leq t+m}^{(1)} = K'_{\leq t+m}^{(1)}$, and $V_{\leq t+m}^{(1)} = V'_{\leq t+m}^{(1)}$, then $h^{(1)} = h'^{(1)}$.

Conclusion. By induction, we have that $h^{(l)} = h'^{(l)}$ of every layer $l = 1, \dots, L$. In particular, since $h = h^{(L)}$ and $h' = h'^{(L)}$, then $h = h^{(L)} = h'^{(L)} = h'$. $\square$

B.3. Proof of Theorem 3.4

Theorem B.3 (Restate of Theorem 3.4). *The time complexity for each agent of LatentMAS is $O((d_h^2 m + d_h m^2 + d_h t m)L)$, where t is the input length of this agent, and m is the length of latent thoughts. In contrast, assuming Theorem 3.1, the time complexity for each agent of the vanilla text-based MAS needs to be $O((d_h^3 m \frac{1}{\log |\mathcal{V}|} + d_h^3 m^2 \frac{1}{\log^2 |\mathcal{V}|} + d_h^2 t m \frac{1}{\log |\mathcal{V}|})L + d_h^2 |\mathcal{V}| m \frac{1}{\log |\mathcal{V}|})$ to achieve the same expressiveness.*

Proof. We analyze the time complexity of our LatentMAS and the vanilla text-based MAS separately.

Time complexity of our method. Recall that a transformer layer consists of two main components: self-attention and feed-forward networks. For a length- $(t + m)$ sequence, the time complexity to compute self-attention for m latent reasoning steps is $O(d_h(t + m)m) = O(d_h(m^2 + tm))$ due to the attention computation between $O(t^2)$ pairs of tokens, and the time complexity to compute feed-forward networks for m latent reasoning steps is $O(d_h^2 m)$ due to matrix-vector multiplication. Since there are L layers, the overall time complexity of our method is

$$O((d_h(m^2 + tm) + d_h^2 m)L). \quad (36)$$

Time complexity of the vanilla text-based MAS. Let m' denote the number of text-based reasoning steps. Similarly to the complexity analysis of our method, the time complexity to compute the hidden embeddings is

$$O((d_h(m'^2 + tm') + d_h^2 m')L). \quad (37)$$

Besides that, due to matrix–vector multiplication and softmax computation, the time complexity to decode hidden embeddings into tokens is

$$O(d_h |\mathcal{V}| m'). \quad (38)$$

Hence, the overall time complexity of the vanilla MAS is

$$O((d_h(m'^2 + tm') + d_h^2 m')L + d_h |\mathcal{V}| m'). \quad (39)$$

Assuming Theorem 3.1, the number of text-based reasoning steps is

$$m' = O\left(\frac{d_h m}{\log |\mathcal{V}|}\right). \quad (40)$$

It follows that the overall time complexity is

$$O((d_h(m'^2 + tm) + d_h^2 m')L + d_h |\mathcal{V}| m') \quad (41)$$

$$= \left(\left(d_h \left(\left(\frac{d_h m}{\log |\mathcal{V}|} \right)^2 + t \left(\frac{d_h m}{\log |\mathcal{V}|} \right) \right) + d_h^2 \left(\frac{d_h m}{\log |\mathcal{V}|} \right) \right) L + d_h |\mathcal{V}| \left(\frac{d_h m}{\log |\mathcal{V}|} \right) \right) \quad (42)$$

$$= O\left(\left(\frac{d_h^3 m^2}{\log^2 |\mathcal{V}|} + \frac{d_h^3 m}{\log |\mathcal{V}|} + \frac{d_h^2 tm}{\log |\mathcal{V}|} \right) L + \frac{d_h^2 |\mathcal{V}| m}{\log |\mathcal{V}|} \right). \quad (43)$$

□

C. Experiment Setups

C.1. Evaluation Details

We introduce all datasets used in our experiments as follows:

Math & Science Reasoning.

- **GSM8K** (Cobbe et al., 2021) is a widely used benchmark of 8.5K grade-school math word problems designed to evaluate multi-step numerical reasoning. Each problem requires decomposing a natural-language description into structured arithmetic steps, making it a standard testbed for assessing chain-of-thought reasoning ability.
- **AIME24** (Maxwell-Jia, 2024) consists of 30 competition-level problems from the 2024 American Invitational Mathematics Examination. These questions span algebra, geometry, number theory, and combinatorics, and require precise numeric answers with typically 1–3 digits, making the benchmark a compact but challenging evaluation of high-school Olympiad-style reasoning.
- **AIME25** (math ai, 2025) provides 30 additional problems from the 2025 AIME exam, maintaining the same answer format and difficulty profile. Compared with AIME24, this benchmark includes more multi-phase derivations and intricate combinatorial constructions, offering a complementary stress test for mathematical robustness.
- **GPQA-Diamond** (Rein et al., 2023) is the most difficult split of the GPQA benchmark with 198 questions, featuring graduate-level multiple-choice questions written by domain experts in physics, biology, and chemistry. The dataset emphasizes conceptual depth, cross-disciplinary reasoning, and the ability to synthesize multi-step scientific arguments under rigorous distractor settings.
- **MedQA** (Yang et al., 2024a) contains real medical licensing exam questions that assess biomedical knowledge, clinical reasoning, and diagnostic decision-making. Problems require integrating textual context with domain-specific medical understanding, making the benchmark a representative testbed for professional-level scientific reasoning.

Commonsense Reasoning.

- **ARC-Easy** (Clark et al., 2018b) consists of grade-school science questions from the AI2 Reasoning Challenge that test foundational factual knowledge and straightforward commonsense reasoning. As a simplified subset of ARC, it serves as a baseline measure of basic scientific understanding without requiring complex multi-step inference.
- **ARC-Challenge** (Clark et al., 2018a) includes the most difficult items from the AI2 Reasoning Challenge. These questions are intentionally adversarial, requiring multi-hop reasoning, causal and counterfactual inference, and systematic elimination of distractor choices. Performance on ARC-Challenge is widely regarded as a strong indicator of robust commonsense reasoning capabilities.

Code Generation.

- **MBPP-Plus** (Liu et al., 2023) extends the original MBPP benchmark with broader input coverage, additional hidden test cases, and stricter execution-based evaluation. Each problem requires generating a self-contained Python function that satisfies a comprehensive unit-test suite, making the benchmark a robust measure of code synthesis reliability and correctness.
- **HumanEval-Plus** (Liu et al., 2023) augments HumanEval with denser, more challenging test suites, significantly increasing the rigor of functional correctness evaluation. The benchmark emphasizes

generalization beyond prompt examples and tests a model’s ability to produce semantically precise, executable Python code under more demanding verification settings.

C.2. Implementation Details

Beyond the experimental setup described in the main paper, we provide additional implementation and evaluation details below.

Software Backend All methods are implemented in Python using PyTorch and HuggingFace Transformers, with an optional vLLM backend for fast decoding and tensor-parallel inference. We use the official chat templates and special tokens such as `<|im_start|>` and `<|im_end|>`.

Evaluation protocol. For all non-coding benchmarks, we report accuracy based on answer matching of the final answer after text normalization (lowercasing, trimming whitespace, and removing extraneous punctuation).

For multiple-choice datasets (GPQA-Diamond, MedQA, ARC-Easy, ARC-Challenge), we first extract the model’s final answer string and then compare it via exact match to the answer letter.

For numeric problems (GSM8k, AIME24, AIME25), we evaluate correctness based on numeric equality: we extract the final predicted answer, parse both prediction and answer into numbers, and mark as correct only if the two values match. Predictions that fail numeric parsing are counted as incorrect.

For code generation tasks (MBPP-Plus and HumanEval-Plus), we evaluate the code by executing unit tests. Specifically, we extract the predicted code from the model’s output, append the ground-truth tests provided by the benchmark, and execute the combined script in a sandboxed environment with a 10-second timeout. A sample is counted as correct if and only if all tests pass without runtime errors.

C.3. Additional Discussions on LatentMAS

Extension to Heterogeneous Agents. For simplicity and training-free purposes, we assume that all agents in LatentMAS share the same shape of transformer layers. To relax this assumption and support heterogeneous agents in practice, one can directly leverage prior studies on layer mapping and ensemble learning (Ainsworth et al., 2022; Wortsman et al., 2022) by introducing a trainable adapter to align and share latent representations across different models.

D. Case Study

Case Study (TextMAS on GSM8K)

Input Question: Debra is monitoring a beehive to see how many bees come and go in a day. She sees 30 bees leave the hive in the first 6 hours, and then she sees $1/2$ that many bees return in the next 6 hours. She sees two times as many bees as she saw first leave the hive fly from the hive and leave in the next 6 hours. Then every bee that left before that hadn't already returned returns to the hive in the next 6 hours. How many bees did Debra see return to the hive in the last 6 hours of the day?

TextMAS Output

Planner Agent: Okay, let's see. I need to figure out how many bees Debra saw return to the hive in the last 6 hours. Let me break down the problem step by step. First, the problem mentions different time intervals: first 6 hours, next 6 hours, then the next 6 hours, and finally the last 6 hours. So there are four 6-hour periods in a day, making up 24 hours total. Let me note each period and what happens in each. Starting with the first 6 hours: Debra sees 30 bees leave the hive. Let me write that down as the first data point. So, Period 1 (0-6 hours): 30 bees leave. Then, the next 6 hours (Period 2: 6-12 hours), she sees half that number return. Half of 30 is 15. So, 15 bees return in Period 2. Next, the third period (Period 3: 12-18 hours), she sees two times as many bees as she saw first leave the hive fly from the hive and leave. The first time she saw 30 bees leave, so two times that is 60. So, 60 bees leave in Period 3. Then, the fourth period (Period 4: 18-24 hours), every bee that left before that hadn't already returned returns to the hive.

Critic Agent: The reasoning plan correctly identifies all four 6-hour periods and reaches the right final conclusion, but the intermediate analysis contains several issues that reduce clarity and efficiency. First, the interpretation of the phrase "before that" is revisited multiple times, even though it simply refers to the event in the previous sentence (the 60 bees leaving), meaning only bees that left in Period 1 are relevant. Second, the plan temporarily entertains the possibility that bees may have left in Period 2, despite the problem statement explicitly describing only returns during this interval. Third, the plan repeatedly questions whether the 15 returning bees in Period 2 originate from the initial 30, even though the text directly states they are "half that many" and therefore unambiguously tied to Period 1. These detours introduce redundancy and confusion. Overall, while the conclusion of 15 returning bees is correct, the path to it is unnecessarily convoluted and would benefit from a more concise timeline-based analysis.

Refiner Agent: I will provide a refined plan based on previous agents' information.

1. Identify the Time Periods and Events. We divide the day into four 6-hour intervals and record the events in each:

- Period 1 (0–6 hours): 30 bees leave the hive.
- Period 2 (6–12 hours): Half of those bees return, i.e., 15 bees.
- Period 3 (12–18 hours): Twice the number of bees that left in Period 1 leave the hive: $2 \times 30 = 60$ bees.
- Period 4 (18–24 hours): Every bee that left *before Period 3* and has not yet returned comes back to the hive.

2. Track Bees Leaving and Returning.

- Period 1: 30 bees leave.
- Period 2: 15 of these 30 bees return, leaving 15 still outside the hive.
- Period 3: 60 new bees leave (these are not relevant for the final return count).
- Period 4: Only bees that left *before Period 3* and have not returned will come back.

3. Determine Which Bees Return in Period 4. The only bees that left before Period 3 are the 30 from Period 1. Of these, 15 returned in Period 2. Thus, the remaining 15 bees from Period 1 return in Period 4.

4. Return the Final Answer.

Solver Agent: Okay, let's see. I need to figure out how many bees Debra saw return to the hive in the last 6 hours. Let me break down the problem step by step.

First, the problem mentions different time intervals: first 6 hours, next 6 hours, then the next 6 hours, and finally the last 6 hours. So there are four 6-hour periods in a day, making up 24 hours total. Let me note each period and what happens in each. Starting with the first 6 hours: Debra sees 30 bees leave the hive. Let me write that down as the first data point. So, Period 1 (0-6 hours): 30 bees leave. Then, the next 6 hours (Period 2: 6-12 hours), she sees half that number return. Half of 30 is 15. So, 15 bees return in Period 2. Next, the third period (Period 3: 12-18 hours), she sees two times as many bees as she saw first leave the hive fly from the hive and leave. The first time she saw 30 bees leave, so two times that is 60. So, 60 bees leave in Period 3. Then, the fourth period (Period 4: 18-24 hours), every bee that left before that hadn't already returned returns to the hive. So, bees that left in Period 1 and Period 2. But some of those bees may have already returned in Period 2.

Final Answer

Debra saw 15 bees return to the hive in the last 6 hours of the day. (✗ Wrong Final Answer)

Case Study (LatentMAS on GSM8K)

Input Question: Debra is monitoring a beehive to see how many bees come and go in a day. She sees 30 bees leave the hive in the first 6 hours, and then she sees $\frac{1}{2}$ that many bees return in the next 6 hours. She sees two times as many bees as she saw first leave the hive fly from the hive and leave in the next 6 hours. Then every bee that left before that hadn't already returned returns to the hive in the next 6 hours. How many bees did Debra see return to the hive in the last 6 hours of the day?

LatentMAS Output

We are given a sequence of events involving bees leaving and returning to the hive over the course of a day, broken into 6-hour intervals.

According to the refiner's plan, we need to track how many bees leave and return during each 6-hour interval, noting that 30 leave first, 15 return next, and 60 more leave afterward. In the final interval, all bees that had left and not yet returned came back, giving a total of 15 remaining from the first group, plus 60 from the third group, for a total of 75 returning bees.

Let's break it down step by step based on the plan:

Step 1: Bees that leave in the first 6 hours. Debra sees 30 bees leave the hive during the first 6-hour period.

Step 2: Bees that return in the next 6 hours. In the second 6-hour period, she sees half as many bees return: $\frac{1}{2} \times 30 = 15$ bees return.

Step 3: Bees that leave in the third 6-hour period. In the next 6 hours, twice the number from Step 1 leave: $2 \times 30 = 60$ bees leave.

Step 4: Bees that return in the final 6 hours. During the final 6 hours, every bee that had left earlier and had not yet returned comes back to the hive.

We track the bees that left and determine which have already returned:

- From the first 6 hours: 30 bees left; 15 returned in Step 2, leaving 15 bees still out.
- From the third 6 hours: 60 bees left; none have returned yet.

Thus, the total number of bees returning in the final 6 hours is $15 + 60 = 75$ bees.

###Final Answer: 75 (✓ *Correct Final Answer*)

To better understand how latent collaboration changes multi-agent reasoning dynamics, we conduct a detailed case study on GSM8K using the Qwen3-14B backbone under the Sequential MAS setting. As shown in the example, TextMAS agents rely on lengthy textual exchanges that often amplify early reasoning errors, misinterpretations by the planner propagate through the critic and refiner, ultimately constraining the solver's search space. In contrast, LatentMAS operates entirely through latent working-memory transfer: each agent receives rich, continuous representations of prior reasoning rather than brittle text, enabling later agents to reinterpret, refine, and correct upstream reasoning without inheriting surface-level mistakes. This latent collaboration leads to more coherent intermediate steps, more stable numerical reasoning, and ultimately yields the correct final answer, where TextMAS fails. The case study illustrates how LatentMAS mitigates error compounding in multi-agent pipelines and demonstrates the qualitative advantage of latent over text-based communication.

E. Prompt Template for LatentMAS

Sequential LatentMAS Prompts on Numeric Tasks (GSM8K / AIME2024 / AIME2025)

System Prompt for All Agents:

You are Qwen, created by Alibaba Cloud. You are a helpful assistant.

Prompt for Planner Agent:

You are a Planner Agent. Given an input question, design a clear, step-by-step plan for how to solve the question. Question: {question}

Your outlined plan should be concise with a few bulletpoints for each step. Do not produce the final answer. Now output your plan to solve the question below:

Prompt for Critic Agent:

You are a Critic Agent to evaluate the correctness of the input plan for the given question and provide helpful feedback for improving the plan. Question: {question}

The plan information is provided in latent KV representation format. Review the plan and question and output: (1) original plan contents (2) constructive feedback on the original plan. Format your response as follows: Original Plan: [Copy the provided Planner Agent's plan here] Feedback: [Your detailed feedback to improve the plan here] Now, output your response below:

Prompt for Refiner Agent:

You are a Refiner Agent to provide a refined step-by-step plan for solving the given question. Question: {question}

You are provided with: (1) latent-format information: a previous plan with feedback (2) text-format information: the input question you need to solve. Based on the input, write a refined and improved plan to solve the question. Make sure your output plan is correct and concise. Now, output your refined plan below:

Prompt for Judger Agent:

You are a helpful assistant. You are provided with latent information for reference and a target question to solve. Target Question: {question}

The latent information might contain irrelevant contents. Ignore it if it is not helpful for solving the target question. Now, reason step by step and output the final answer inside \boxed{YOUR_FINAL_ANSWER}:

Sequential LatentMAS prompts for multiple-choice tasks (ARC-E, ARC-C, GPQA, MedQA)

System Prompt for All Agents:

You are Qwen, created by Alibaba Cloud. You are a helpful assistant.

Prompt for Planner Agent:

You are a Planner Agent. Given an input question, design a clear, step-by-step plan for how to solve the question. Question: {question}

Your outlined plan should be concise with a few bulletpoints for each step. Do not produce the final answer. Now output your plan to solve the question below:

Prompt for Critic Agent:

You are a Critic Agent to evaluate the correctness of the input plan for the given question and provide helpful feedback for improving the plan. Question: {question}

The plan information is provided in latent KV representation format. Review the plan and question and output: (1) original plan contents (2) constructive feedback on the original plan. Format your response as follows: Original Plan: [Copy the provided Planner Agent's plan here] Feedback: [Your detailed feedback to improve the plan here] Now, output your response below:

Prompt for Refiner Agent:

You are a Refiner Agent to provide a refined step-by-step plan for solving the given question. Question: {question}

You are provided with: (1) latent-format information: a previous plan with feedback (2) text-format information: the input question you need to solve. Based on the input, write a refined and improved plan to solve the question. Make sure your output plan is correct and concise. Now, output your refined plan below:

Prompt for Judger Agent:

You are a helpful assistant. You are provided with latent information for reference and a target question to solve. Target Question: {question}

The latent information might contain irrelevant contents. Ignore it if it is not helpful for solving the target question. Your final answer must be selected from A,B,C,D. For example \boxed{A}. Do not add any other contents inside the box. Now, reason step by step and output the final answer inside \boxed{YOUR_FINAL_ANSWER}:

Sequential LatentMAS prompts for python coding tasks (MBPP-Plus, HumanEval-Plus)

System Prompt for All Agents:

You are Qwen, created by Alibaba Cloud. You are a helpful assistant.

Prompt for Planner Agent:

You are a Planner Agent. Given an input question, design a clear, step-by-step plan for how to solve the question. Question: {question}

Your outlined plan should be concise with a few bulletpoints for each step. Do not produce the final answer. Now output your plan to solve the question below:

Prompt for Critic Agent:

You are a Critic Agent to evaluate the correctness of the input plan for the given question and provide helpful feedback for improving the plan. Question: {question}

The plan information is provided in latent KV representation format. Review the plan and question and output: (1) original plan contents (2) constructive feedback on the original plan. Format your response as follows: Original Plan: [Copy the provided Planner Agent's plan here] Feedback: [Your detailed feedback to improve the plan here] Now, output your response below:

Prompt for Refiner Agent:

You are a Refiner Agent to provide a refined step-by-step plan for solving the given question. Question: {question}

You are provided with: (1) latent-format information: a previous plan with feedback (2) text-format information: the input question you need to solve. Based on the input, write a refined and improved plan to solve the question. Make sure your output plan is correct and concise. Now, output your refined plan below:

Prompt for Judger Agent:

You are a helpful assistant. You are provided with latent information for reference and a target question to solve. Target Question: {question}

The latent information might contain irrelevant contents. Ignore it if it is not helpful for solving the target question. You must reason step-by-step to solve the **provided Target Question** without outputting other irrelevant information. You must put all python code as self-contained Python function in markdown code blocks. For example

```
'''python
import math
def add(a, b):
    return a + b'''
```

Do not add any other contents inside the markdown code block. Now, reason step by step and output the final answer inside '''python YOUR_PYTHON_CODE''':

Hierarchical LatentMAS prompts for numeric-answer tasks (GSM8K, AIME2024, AIME2025)

System prompt for All Agents:

You are Qwen, created by Alibaba Cloud. You are a helpful assistant.

Prompt for Math Agent:

You are a math agent. Given the input question, reason step-by-step and put the final answer inside `\boxed{YOUR_FINAL_ANSWER}`. Question: {question}

Your response:

Prompt for Science Agent:

You are a science agent. Given the input question, reason step-by-step and put the final answer inside `\boxed{YOUR_FINAL_ANSWER}`. Question: {question}

Your response:

Prompt for Code Agent:

You are a code agent. Given the input question, reason step-by-step and put the final answer inside `\boxed{YOUR_FINAL_ANSWER}`. Question: {question}

Your response:

Prompt for Task Summarizer Agent:

You are a task summarizer. Given the input question and responses from previous agents as reference, reason step-by-step and put the final answer inside `\boxed{YOUR_FINAL_ANSWER}`.

Question: {question}

Your response:

Hierarchical LatentMAS prompts for multiple-choice tasks (ARC-E, ARC-C, GPQA, MedQA)

System Prompt for All Agents:

You are Qwen, created by Alibaba Cloud. You are a helpful assistant.

Prompt for Math Agent:

You are a math agent. Given the input question, reason step-by-step and put the final answer inside `\boxed{YOUR_FINAL_ANSWER}`. Your final answer must be selected from A,B,C,D. For example `\boxed{A}`. Do not add any other contents inside the box. Question: {question}

Your response:

Prompt for Science Agent:

You are a science agent. Given the input question, reason step-by-step and put the final answer inside `\boxed{YOUR_FINAL_ANSWER}`. Your final answer must be selected from A,B,C,D. For example `\boxed{A}`. Do not add any other contents inside the box. Question: {question}

Your response:

Prompt for Code Agent:

You are a code agent. Given the input question, reason step-by-step and put the final answer inside `\boxed{YOUR_FINAL_ANSWER}`. Your final answer must be selected from A,B,C,D. For example `\boxed{A}`. Do not add any other contents inside the box. Question: {question}

Your response:

Prompt for Task Summarizer Agent:

You are a task summarizer. Given the input question and responses from previous agents as reference, reason step-by-step and put the final answer inside `\boxed{YOUR_FINAL_ANSWER}`. Your final answer must be selected from A,B,C,D. For example `\boxed{A}`. Do not add any other contents inside the box. Question: {question}

Your response:

Hierarchical LatentMAS prompts for python coding tasks (MBPP-Plus, HumanEval-Plus)**System Prompt for All Agents:**

You are Qwen, created by Alibaba Cloud. You are a helpful assistant.

Prompt for Math Agent:

You are a math agent. Given the input question, reason step by step and provide an efficient and self-contained Python function that solves the following problem in a markdown code block. You must put all python code as self-contained Python function in markdown code blocks. For example

```
'''python
import math
def add(a, b):
return a + b'''
```

. Do not add any other contents inside the markdown code block.
 Question: {question}
 Your response:

Prompt for Science Agent:

You are a science agent. Given the input question, reason step by step and provide an efficient and self-contained Python function that solves the following problem in a markdown code block.

You must put all python code as self-contained Python function in markdown code blocks. For example

```
'''python
import math
def add(a, b):
return a + b'''
```

. Do not add any other contents inside the markdown code block.
 Question: {question}
 Your response:

Prompt for Code Agent:

You are a code agent. Given the input question, reason step by step and provide an efficient and self-contained Python function that solves the following problem in a markdown code block.

You must put all python code as self-contained Python function in markdown code blocks. For example

```
'''python
import math
def add(a, b):
return a + b'''
```

. Do not add any other contents inside the markdown code block.
 Question: {question}
 Your response:

Prompt for Task Summarizer Agent:

You are a task summarizer. Given the input question and responses from previous agents as reference, reason step by step and provide an efficient and self-contained Python function that solves the following problem in a markdown code block.

You must put all python code as self-contained Python function in markdown code blocks. For example

```
'''python
import needed_library
def FUNC_NAME(a, b):
return a + b'''
```

. Do not add any other contents inside the markdown code block.
 Input Question: {question}
 Your response: